

Radical Denesting with `Strad.wl`

A unified analysis of three independent reviews,
an experimental evaluation in a live Wolfram kernel,
and a corrected implementation

5 September 2026

Abstract

`Strad.wl` is a 585-line Wolfram Language program that tries to denest algebraic radicals such as $\sqrt{5+2\sqrt{6}} = \sqrt{2} + \sqrt{3}$ by searching for a multiplier m that makes the minimal polynomial of $(m\beta^q)^{1/q}$ share a low-degree factor with the binomial $X^q - m\beta^q$ over an algebraic coefficient field. Three independent reviews of the program were produced earlier; all three were purely static and none executed the code. This document compares the three reviews, reconstructs the algorithm, proves the mathematics that justifies it, and then does what the reviews could not: it runs the program in Wolfram 15.0.1 on more than 400 inputs (classical identities, the reviews’ counterexamples, symbolic input, higher-order roots, and randomized families), runs the reviews’ own never-executed regression suites, and measures timings.

The experiments confirm the central defect predicted by all three reviews: the program certifies candidates against the principal root of β^q instead of against β , so 39 of 40 random inputs of the form $(p + qi\sqrt{c})^{3/2}$ and 17 of 40 random products of complex square roots come back with the wrong value. The experiments also uncover facts that no review established: the program changes the value of purely numeric input through its custom factorizer; its acceptance test routinely relies on `PossibleZeroQ` answering “unable to decide, assuming zero”; the mis-typed prime comparator reverses the prime list, which silently disables the prime-pruning loop and hides an out-of-range slice that only appears once the comparator is fixed; list input yields nonsense; and several “successful” denestings are far uglier than the input. On the positive side, the program denests every classical identity we tried in under a second, and its denominator rationalizer is sound.

A corrected package, `StradFixed.wl`, retains the architecture but certifies every candidate against the original value with exact algebra, makes the factorizer branch-safe, restricts factoring to exact numeric subexpressions, fixes the comparator, slice, cap, and empty-list defects, adds budgets, a visited set, private symbols, a quadratic-surd fast path, candidate polishing and an explicit improvement criterion. On the same battery it produces no wrong value, no kernel message, and finds several denestings the original misses, including the depth-three Borodin–Fagin–

Hopcroft–Tompkins example $\sqrt{16 - 2\sqrt{29} + 2\sqrt{55 - 10\sqrt{29}}} = \sqrt{5} + \sqrt{11 - 2\sqrt{29}}$. The document closes with a prioritized list of future improvements.

Evidence. All executions reported here were performed in *15.0.1 for Microsoft Windows (64-bit)* (July 2, 2026) through `wolframscript`. Every table of results is generated mechanically from the recorded runs. Statements about the reviews cite their finding identifiers.

Contents

1	Scope, material, and method	3
1.1	The object under analysis	3
1.2	The three reviews	3
1.3	What this document adds	3
1.4	Conventions	4
2	How the code works	4
2.1	Function map	4
2.2	The wrapper, step by step	4
2.3	The engine	5
2.4	Two actual traces	6
2.5	The helpers	7
3	Mathematical foundations	8
3.1	Why the GCD step is meaningful	8
3.2	Why the roots-of-unity orbit is right, and why the filter is wrong	8
3.3	The rationalizer is sound	9
3.4	When power identities are valid	9
3.5	Quadratic surds	9
3.6	How many multipliers a batch contains	10
4	The three reviews compared	10
4.1	Coverage matrix	10
4.2	Where the reviews agree	10
4.3	Where they differ	10
4.4	What the reviews could not know	11
4.5	The reviews' own test suites, executed	11
5	Experiments	12
5.1	Setup	12
5.2	The original program	12
5.3	Randomized families	23
5.4	The reviews' regression suites	24
5.5	The reviews' proposed kernels	24
5.6	The corrected version	24
6	Unified catalogue of defects and shortcomings	29

6.1	Wrong values	29
6.2	Unsafe acceptance and validation	30
6.3	Search control	31
6.4	Output quality	32
6.5	Interfaces and session hygiene	33
6.6	What is <i>not</i> a defect	34
7	The corrected version	34
7.1	Interface	34
7.2	Change list	35
7.3	What was deliberately kept	36
7.4	Guarantees and their limits	36
7.5	Cost	36
8	Future improvements	36
8.1	Algorithmic	36
8.2	Search engineering	37
8.3	Testing and quality	38
9	Conclusion	38
A	The original source, with line numbers	41
B	The corrected source	50
C	Experiment harness	59

1 Scope, material, and method

1.1 The object under analysis

The file `src/original/Strad.wl` (585 physical lines, SHA-256 693a2825...cccc93, byte-identical to the copies shipped with the three reviews) defines a radical denester for the Wolfram Language. Its entry point `Strad[expr]` suppresses diagnostics and calls `DenestRadicals3`, which factors the input, marks every subexpression of the form $\text{radicand}^{p/q}$ whose radicand is an explicit algebraic number, groups some products of markers, and hands each marked value to the search engine `denest11`. The engine chooses a root order q , forms $\rho = \beta^q$, and searches over multipliers m for a lower-degree factor of $X^q - m\rho$ over an algebraic coefficient field. Helpers implement a minimal-polynomial based denominator rationalizer, nesting-depth annotations, and a custom factorizer. Appendix A reproduces the file with line numbers; all “[lines \$n\$](#) ” references point there.

1.2 The three reviews

Three independent reviews were written on the same day, 5 September 2026, and are kept under `reports/review-1`, `review-2` and `review-3`. We refer to them as R1, R2 and R3. Table 1 summarizes them. They agree on the architecture, on the most serious defect (the principal-root reset), and on the broad repair strategy. They differ in the defects they add, in their emphasis, and in a few predictions that the live experiments below refine or overturn. Crucially, *none of the three ran the program*: each states explicitly that its Wolfram tests were not executed and that its proposed replacement component is unexecuted.

	R1	R2	R3
Length	38 pages, 14 findings F01–F14	34 pages, 16 findings B1–B10, H1–H5, M1	≈30 pages, 12 findings F1–F12
Executed checks	45 SymPy/mpmath checks	SymPy/mpmath script	26 SymPy checks
Wolfram execution	none	none	none
Proposed code	<code>VerifiedMultiplierTrial</code> (one trial)	<code>CertifiedMultiplierStep</code> (one trial)	<code>TryDenestCandidate</code> + cost policy
Regression file	14 <code>VerificationTests</code>	15 tests (<code>.wlt</code>)	20 tests (<code>.wlt</code>) + runner
Distinctive content	complex-product wrapper example; discriminant/index discussion; “what should not be misdiagnosed”	marker <code>f</code> and variable <code>x</code> collisions; equal-degree gate counterexample; <code>List @@ Expand</code> head issue	stale-length slice; quadratic-surd fast path; explicit cost tuple; <code>combine</code> unit-entry gap

Table 1: The three reviews at a glance.

1.3 What this document adds

1. A single reconstruction of how the code works, with actual search traces (Section 2), and proofs of the statements that justify or refute its steps (Section 3).
2. A finding-by-finding comparison of the three reviews, including the places where a review over- or under-stated a defect (Section 4).

3. Executed experiments: 90 hand-chosen probes of the original program, 20 helper-level probes, 340 randomized inputs in ten structured families, the reviews' own regression suites, and the same batteries against the corrected version (Section 5).
4. A unified, numbered catalogue of 27 defects and shortcomings, each with its status (confirmed, refined, refuted, or theoretical), its source lines, and the reviews that reported it (Section 6).
5. A corrected implementation, `src/corrected/StradFixed.wl`, that keeps the original architecture and search heuristics but makes correctness independent of them (Section 7, Appendix B).
6. A prioritized list of future improvements (Section 8).

1.4 Conventions

Throughout, β denotes the exact algebraic number handed to the search engine (the “problem”), $q \geq 2$ the chosen root order, $\rho = \beta^q$ the “radicand”, m a multiplier, and P_m the minimal polynomial over $\mathbb{Q}$ of the principal value $(m\rho)^{1/q}$. Rational powers of complex numbers have their Wolfram Language meaning: $z^{p/q} = \exp(\frac{p}{q} \log z)$ with the principal logarithm, so $\text{Arg } z^{1/q} \in (-\pi/q, \pi/q]$. The syntactic radical depth $\text{rdepth}(E)$ is the maximal number of `Power[_ , _Rational]` nodes on a root-to-leaf path of the expression tree of E ; it is a property of the representation, not of the number.

Experiment identifiers such as A02 or D06 refer to the rows of the tables in Section 5. Every result quoted in the text is copied from those mechanically generated tables or from the recorded logs.

2 How the code works

2.1 Function map

Table 2 lists the twelve top-level definitions and the two local functions. The three layers are an orchestration layer (`Strad`, `DenestRadicals3`), the search engine (`denest11` with its local `trymultiplier` and `newmultipliers`), and helpers.

2.2 The wrapper, step by step

Let E be the input. `DenestRadicals3[E, alllevels, func]` proceeds as follows (lines 5–76).

1. **Factor.** $E \mapsto \text{Factorc}[E]$. This happens *before* any algebraicity test, on the whole expression, symbolic parts included.
2. **Mark.** Every subexpression matching `radicand_?AlgebraicQ^pow_Rational` is replaced by a marker `f[value, pow]`. With `alllevels != True` this is a single `ReplaceAll`, which does not descend into the replaced parts, so inner radicals stay inside the outer marker; with `alllevels == True` it is `ReplaceRepeated`, which keeps marking inside markers and produces nested markers such as `f[Sqrt[55 - 10 f[Sqrt[29]]]]` (observed in the trace of experiment A30). The `value` is a *normalized* radicand: numeric content is split off with `FactorTermsList`, the rest is factored again, Gaussian integer content is decomposed, factors that contain a sum or a complex number with nonzero real part are multiplied together, factors with numerically negative real part (100 digits) are negated with a compensating sign

Function	Lines	Role
Strad	1–3	Unprotects Print at load time; calls the wrapper with Print dynamically replaced by Null & .
DenestRadicals3	5–76	Factors the input, marks $\text{rad}^{p/q}$ subexpressions with a local head f , normalizes and groups them, calls the engine, substitutes, simplifies.
denest11	79–441	Chooses q , builds initial multipliers, runs the multiplier search with a history of degrees and a work stack.
trymultiplier	107–155	One trial: minimal polynomial, degree gate, polynomial GCD, roots-of-unity orbit, PossibleZeroQ filter.
newmultipliers	159–211	New batch of multipliers from the discriminant of a recorded minimal polynomial, or the product of two multipliers.
RationalizeDenominator	444–451	$1/d = -Q(0)/P(0)$ where P is the minimal polynomial of d and $P = (X - d)Q$.
AlgebraicQ	454	Element[num, Algebraics] .
nestdepths, nestdepthsort, maxnestdepth	457–498	Annotate the expression tree with a nesting height for a predicate; group by height; extract the nodes of maximal height.
ComplexitySort	501–504	Sort by LeafCount , then by absolute value.
Factorc, fullfactor, combine	507–584	Custom factorizer: encode products, powers and sums as nested lists, extract common bases from sums, decode.

Table 2: Function map of **Strad.wl**.

accumulator, each factor is rationalized, and the product is simplified (lines 12–41). This step preserves the value by construction.

3. **Group.** Two markers in one product whose radicands both contain a **Power** are merged into one marker holding the product of the two powers (lines 43–47); markers become one-argument **f[rad^pow]** (lines 48); two one-argument markers whose numerical values have nonzero real and imaginary parts are merged (lines 49–54). All three rules keep the product inside the marker, so they are also value preserving; but they create problems β that are products or non-principal powers, which matters in Section 3.2.
4. **Solve.** In single-level mode each distinct marker **f[v]** becomes a rule **f[v] -> func[v]** and the result is **Simplifyd** (lines 58–61). In all-level mode a loop repeatedly picks a marker that contains no other marker, solves it, and substitutes the answer into the remaining markers (lines 62–73); this is a hand-written innermost-first resolution.

Strad[args] simply wraps this in **Block[{Print = (Null &)}, ...]**. Note what is *not* suppressed: kernel messages. Section 5.2 shows that ordinary successful calls emit **PossibleZeroQ::ztest1** and **N::meprec**.

2.3 The engine

denest11[problem, forcedmultipliers] (lines 79–441) works on one exact value β .

1. **Gate.** If β contains no **Power** whose base contains a **Power**, return β (lines 84–88).
2. **Root order.** q = lcm of the exponent denominators of the rational-power nodes of maximal nesting height (lines 90–92); $\rho = \beta^q$ (lines 94). A second quantity **extrapower** is computed and printed but never used (lines 95–103).
3. **Initial multipliers.** Unless a list is supplied: expand the rationalized ρ , strip rational coefficients from its terms, replace complex atoms by $\pm i$, and for each remaining radical factor $b^{n/d}$ form the complementary factor $b^{(d-n)/d}$; prepend 1 (lines 224–249). For $\rho = 5 + 2\sqrt{6}$ this gives $\{1, \sqrt{6}\}$; for $\rho = \sqrt[3]{2} - 1$ it gives $\{1, 2^{2/3}\}$.
4. **Trial** (**trymultiplier**, lines 107–155). For a multiplier m : $P_m = \text{MinPoly}((m\rho)^{1/q})$, $d_m = \deg P_m$. If $d_m < d_{\text{best}}$ or $\gcd(d_m, d_{\text{best}}) < d_{\text{best}}$, compute $G_m = \gcd(P_m(X), X^q - m\rho)$ with **Extension** $\rightarrow$ **Automatic**. If $0 < \deg G_m < q$: solve $G_m = 0$, divide each root by $m^{1/q}$, multiply by all q -th roots of unity, keep the candidates c with **PossibleZeroQ**[**c - radicand^{outerpower}**], take the first, and report a *solution*. Otherwise report *progress* if the degree decreased. If the gate failed, report progress when the degree is different from the best and no worse degree is known yet (lines 147–152).
5. **Search control** (lines 251–432). A **history** of records $\{m, P_m, d_m\}$ is kept sorted by decreasing degree, seeded with $\{0, \infty, \infty\}$. A **multiplierstack** of batches is processed in order; batches are either concrete lists or deferred calls to **newmultipliers**. Progress on a full-set batch schedules a new batch later; progress on a partial batch splits the current batch and inserts the new one immediately. A solution stops the search when it is the first finite-degree record or when the degree dropped by a factor $\geq q$; otherwise the search continues for “additional denesting”. After the first pass, products of the promising initial multipliers are formed by **Distribute** and appended (lines 398–428).
6. **New multipliers** (**newmultipliers**, lines 159–211). In the usual branch, partially factor $\text{Disc}(P_{\text{smaller}})$, drop composite cofactors, sort the primes (with the mis-typed comparator, see U5), trim primes separated by a ratio above 100, take a prefix of length $\max(5, \lceil \log 1000 / \log q \rceil)$, and return $m_{\text{smaller}} \cdot (\text{Divisors}((\prod p_i)^{q-1}) \cup \{p_i\}) \setminus \{1\}$ sorted by complexity. In the other branch (degrees with a smaller common divisor) return the product of the two multipliers.

2.4 Two actual traces

The following is the diagnostic output of **DenestRadicals3**[**Sqrt**[**5 + 2 Sqrt**[**6**]]] in the kernel (Section 5.1). The first two multipliers give degree 4 without a proper GCD; the discriminant $\text{Disc}(P_1) = 2^{14}3^2$ yields the batch $\{2, 3, 6\}$ and $m = 2$ succeeds.

```
{f[Sqrt[5 + 2*Sqrt[6]]]}
problem  Sqrt[5 + 2*Sqrt[6]]
outer root: 2  extra root: 1
multiplier count: 2  multipliers: {1, Sqrt[6]}
multiplier: 1  minpoly degree: 4
adding new multipliers from: 1
multiplier: Sqrt[6]  minpoly degree: 4
adding new multipliers from: Sqrt[6]
multiplier count: 3  from: {1, 4}
multiplier: 2  minpoly degree: 2
success: {2, 2, {{2, 1}}}
history: {{0, Infinity}, {1, 4}, {2, 2}}
```

[Sqrt\[2\]](#) + [Sqrt\[3\]](#)

For Ramanujan's $\sqrt[3]{\sqrt{2}-1}$ the engine needs $m = 9$, found as the fifth element of the discriminant batch:

```
problem (-1 + 2^(1/3))^(1/3)
outer root: 3  extra root: 1
multiplier count: 2  multipliers: {1, 2^(2/3)}
multiplier: 1  minpoly degree: 9
adding new multipliers from: 1
multiplier: 2^(2/3)  minpoly degree: 9
adding new multipliers from: 2^(2/3)
multiplier count: 8  from: {1, 9}
multiplier: 2  minpoly degree: 9
multiplier: 3  minpoly degree: 9
multiplier: 4  minpoly degree: 9
multiplier: 6  minpoly degree: 9
multiplier: 9  minpoly degree: 3
success: {9, 3, {{3, 2}}}
history: {{0, Infinity}, {1, 9}, {9, 3}}
(1 - 2^(1/3) + 2^(2/3))/3^(2/3)
```

Indeed $(1 - \sqrt[3]{2} + \sqrt[3]{4})^3 = 9(\sqrt[3]{2} - 1)$, so the multiplier 9 turns the degree-9 number $(\sqrt[3]{2} - 1)^{1/3}$ into the degree-3 number $1 - \sqrt[3]{2} + \sqrt[3]{4}$, whose minimal polynomial has a linear common factor with $X^3 - 9(\sqrt[3]{2} - 1)$ over $\mathbb{Q}(\sqrt[3]{2})$.

2.5 The helpers

Rationalizer. `RationalizeDenominator` ([lines 444–451](#)) is mathematically sound ([Proposition 3.4](#)) but uses the symbol `x` both as the polynomial variable and, through `/.` `x -> 0`, as a substitution target on the numerator ([U11](#)).

Nesting helpers. `nestdepths` returns a nested list $\{E, \text{child annotations} \dots, h(E)\}$ where $h(E) = 1_{P(E)} + \max_c h(c)$; `maxnestdepth` prunes everything below the maximum height. With the predicate “is a rational power” this is `rdepth`. The encoding mixes data lists with structure lists and is the reason list-valued input misbehaves ([U15](#)).

Custom factorizer. `fullfactor` starts from `FactorList` and repeatedly (at most five passes) rewrites each {factor, exponent} pair: a power $\{a^b, c\}$ becomes $\{a, bc\}$; a composite rational becomes its prime factorization; a sum is factored term by term by `combine`, which pulls bases common to all terms out of the sum with their minimal exponent, later multiplied by the outer exponent. `Factorrc` decodes the nested lists with five shape rules. The two rewrites in italics are exactly the identities $(a^b)^c = a^{bc}$ and $(FS)^p = F^p S^p$, applied without any branch condition ([U2](#)).

3 Mathematical foundations

This section collects the statements that justify the engine, the statements that show where it goes wrong, and the two identities used by the corrected version. All three reviews prove variants of Propositions 3.1–3.4; the power-identity conditions of Proposition 3.5 and the quadratic-surd formula of Proposition 3.6 are stated in R3 in a weaker form.

3.1 Why the GCD step is meaningful

Proposition 3.1. *Let $\rho \neq 0$ and $m \neq 0$ be algebraic numbers, $q \geq 2$, and let $K \supseteq \mathbb{Q}$ be the field generated by the coefficients used by *PolynomialGCD* with *Extension -> Automatic* (it contains $m\rho$). Let P_m be the minimal polynomial over $\mathbb{Q}$ of $\alpha = (m\rho)^{1/q}$ and $G_m = \gcd_{K[X]}(P_m, X^q - m\rho)$. Then $1 \leq \deg G_m \leq q$, and every root of G_m is a q -th root of $m\rho$.*

Proof. α is a common root, so the minimal polynomial of α over K divides both polynomials and hence their GCD; thus $\deg G_m \geq 1$. G_m divides the degree- q binomial, which gives the upper bound and the root property. $\square$

A GCD of degree $e < q$ is an equation of degree e over K for some q -th root of $m\rho$. In the best case $e = 1$ and that root is an element of K written without the outer radical. Nothing in the proposition says that solving G_m yields a *simpler radical expression*: the roots may involve *Root* objects or nested radicals of their own (experiments Q01–Q08 show candidates containing $\sqrt{9 - 6\sqrt{2}}$, which is itself denestable).

3.2 Why the roots-of-unity orbit is right, and why the filter is wrong

Proposition 3.2. *Let $\beta \neq 0$, $q \geq 2$, $\rho = \beta^q$, $m \neq 0$, and let r be any root of G_m . Then $\{r m^{-1/q} \zeta_q^k : 0 \leq k < q\}$, $\zeta_q = e^{2\pi i/q}$, is exactly the set of q -th roots of ρ ; in particular it contains β .*

Proof. $r^q = m\rho$ and $(m^{1/q})^q = m$ for any choice of $m^{1/q}$, so $(r/m^{1/q})^q = \rho$. The q numbers are distinct since $\rho \neq 0$, and $X^q - \rho$ has exactly q roots. $\square$

No identity of the form $(m\rho)^{1/q} = m^{1/q}\rho^{1/q}$ is used. The code builds precisely this orbit (lines 121–124); the orbit is a strength of the program. The defect is the selection:

```
Select[possibilities, PossibleZeroQ[# - radicand^outerpower] &]
```

compares against $\rho^{1/q} = (\beta^q)^{1/q}$, not against β .

Proposition 3.3. *For $\beta = re^{i\theta} \neq 0$ with $\theta = \text{Arg } \beta \in (-\pi, \pi]$ and an integer $q \geq 2$,*

$$(\beta^q)^{1/q} = \beta e^{-2\pi i k/q}, \quad k \in \mathbb{Z} \text{ with } q\theta - 2\pi k \in (-\pi, \pi].$$

Hence $(\beta^q)^{1/q} = \beta$ if and only if $-\pi/q < \text{Arg } \beta \leq \pi/q$.

Proof. $\text{Arg}(\beta^q) = q\theta - 2\pi k$ by definition of the principal argument; take the principal q -th root. $\square$

Every input to the engine that is itself a principal root ($\beta = \rho^{1/q}$ literally) lies in the sector, which is why the classical examples work. The wrapper, however, sends products of markers and powers $\text{rad}^{p/q}$ with $p > 1$ to the engine; those need not lie in the sector. Example: $\beta = (-1 + 2i\sqrt{2})^{3/2} = (1 + i\sqrt{2})^3 = -5 + i\sqrt{2}$ has $\text{Arg } \beta > \pi/2$, so $(\beta^2)^{1/2} = 5 - i\sqrt{2} = -\beta$, and the program returns $5 - i\sqrt{2}$ (experiment C02). The randomized family F5 shows that this is the typical, not the exceptional, outcome for such inputs.

3.3 The rationalizer is sound

Proposition 3.4. *Let $d \neq 0$ be algebraic with minimal polynomial $P \in \mathbb{Q}[X]$ (any nonzero rational multiple, not necessarily monic), and let $P(X) = (X - d)Q(X)$. Then $P(0) \neq 0$ and $1/d = -Q(0)/P(0)$.*

Proof. P is irreducible of degree ≥ 1 with root $d \neq 0$, so $P(0) \neq 0$. Evaluating $P(X) = (X - d)Q(X)$ at 0 gives $P(0) = -dQ(0)$. $\square$

This is exactly [lines 447–450](#). Experiments R01–R09 confirm the helper on quadratic, quartic, complex, cubic, and nested denominators.

3.4 When power identities are valid

Proposition 3.5. *For nonzero complex a, F, S and real exponents:*

1. $(a^b)^c = a^{bc}$ holds whenever $c \in \mathbb{Z}$, or $b \in \mathbb{R}$ with $-1 < b \leq 1$.
2. Let $p = n/q$ in lowest terms with $q \geq 2$. Then $(FS)^p = F^p S^p$ if and only if $\text{Arg } F + \text{Arg } S \in (-\pi, \pi]$. In particular it holds whenever $F > 0$ or $S > 0$.

Proof. (1) If $c \in \mathbb{Z}$, $(a^b)^c = \exp(c \log(a^b))$ and $\exp(c \cdot 2\pi i k) = 1$ absorbs the branch. If $-1 < b \leq 1$ then $\text{Im}(b \log a) = b \text{Arg } a \in (-\pi, \pi]$, so $\log(a^b) = b \log a$ exactly and $(a^b)^c = \exp(cb \log a) = a^{bc}$. (2) $\log(FS) = \log F + \log S - 2\pi i k$ with $k \in \{-1, 0, 1\}$ chosen so that the imaginary part lies in $(-\pi, \pi]$; $k = 0$ iff $\text{Arg } F + \text{Arg } S \in (-\pi, \pi]$. Then $(FS)^p = F^p S^p \exp(-2\pi i k p)$, and $\exp(-2\pi i k n/q) = 1$ iff $q \mid kn$, i.e. iff $k = 0$ since $\gcd(n, q) = 1$ and $|k| \leq 1 < q$. $\square$

The custom factorizer applies (1) with rational c and unrestricted b ([lines 527–531](#)) and (2) with rational p and unrestricted F ([lines 571–583](#)). Counterexamples for (1): $\sqrt{z^2} \mapsto z$ (E01). For (2): $\sqrt{-x-y} \mapsto i\sqrt{x+y}$ (E03), and, entirely numerically, $F = -1$, $S = \sqrt{2} + (-1)^{1/3}$ with $\text{Arg } F + \text{Arg } S = \pi + \pi/9 > \pi$, where the program returns $i\sqrt{S} = -\sqrt{-S}$ (J01). The corrected factorizer applies (1) only under the stated condition and (2) only when $p \in \mathbb{Z}$, $F > 0$ or $S > 0$.

3.5 Quadratic surds

Proposition 3.6. *Let $a, b, c \in \mathbb{Q}$ with $a > 0$, $b \neq 0$, $c > 0$ not a square, and suppose $\Delta = a^2 - b^2 c = \delta^2$ with $\delta \in \mathbb{Q}_{\geq 0}$. Put $u = (a + \delta)/2$ and $v = (a - \delta)/2$. Then $u \geq v \geq 0$ and*

$$\sqrt{a + b\sqrt{c}} = \sqrt{u} + \text{sgn}(b)\sqrt{v},$$

where both sides are principal square roots. If $a < 0$ and $a + b\sqrt{c} < 0$ then $\sqrt{a + b\sqrt{c}} = i\sqrt{-a - b\sqrt{c}}$ and the formula applies to $-a, -b$.

Proof. $a \geq \delta$ because $a^2 = \Delta + b^2c \geq \Delta$, so $v \geq 0$. The square of the right side is $u+v+2\operatorname{sgn}(b)\sqrt{uv} = a + \operatorname{sgn}(b)\sqrt{a^2 - \Delta} = a + \operatorname{sgn}(b)|b|\sqrt{c} = a + b\sqrt{c}$, and the right side is $\geq \sqrt{u} - \sqrt{v} \geq 0$, so it is the principal root. The negative case is $\sqrt{-w} = i\sqrt{w}$ for $w > 0$. $\square$

This is the classical criterion (one half of Landau’s theorem for quadratic surds); the corrected version uses it both as a fast path and to clean inner surds that appear inside candidates, e.g. $\sqrt{9 - 6\sqrt{2}} = \sqrt{6} - \sqrt{3}$.

3.6 How many multipliers a batch contains

Proposition 3.7. *If t distinct primes are selected, the batch built from Divisors $((\prod_{i \leq t} p_i)^{q-1})$ contains exactly q^t elements before the unit is dropped.*

Proof. Each divisor is $\prod p_i^{e_i}$ with independent $0 \leq e_i \leq q - 1$. $\square$

The code selects $t = \min(\#\text{primes}, \max(5, \lceil \log 1000 / \log q \rceil))$ (lines 190–194), so the nominal cap of 1000 is not a cap: $t = 10$ for $q = 2$ gives 1023 candidates, and the forced minimum $t = 5$ gives $q^5 - 1$ for large q (99 999 for $q = 10$; experiment N06 materializes exactly that list). The corrected version chooses the largest t with $q^t - 1 \leq \text{cap}$ in integer arithmetic.

4 The three reviews compared

4.1 Coverage matrix

Table 3 maps every finding of every review onto the unified catalogue of Section 6. A dot means the review reports the item as a finding; a circle means it is mentioned in passing. The last column gives the experimental status established in Section 5.

4.2 Where the reviews agree

All three reviews reconstruct the same algorithm, prove the same three propositions (nonconstant GCD, orbit coverage, principal-root reset), single out U1 and U2 as the critical defects, recognize the rationalizer as sound, and recommend the same repair order: certify against the original value, remove or guard unsafe rewrites, validate inputs and intermediate results, then tune heuristics. Each ships a one-multiplier “certified trial” component with the same shape (validate, minimal polynomial, GCD, orbit, exact certification, structured result). The experiments vindicate this consensus: every wrong value observed is a consequence of U1 or U2, and the corrected version’s certification layer eliminates all of them.

4.3 Where they differ

- R2 is the only review that notices the two symbol collisions (U10, U11) and the equal-degree gate (U12); all three predictions are confirmed exactly (M01, R05, D06).
- R3 is the only review that notices the stale-length slice (U6) and proposes the quadratic-surd fast path and an explicit cost tuple; the corrected version adopts both. Its F9 remark about `MemberQ[... , Infinity]` excluding level 0 is correct as a reading of the code but moot for

exact numeric input, because the kernel auto-simplifies $(a^{1/2})^{1/3}$ and similar nested powers before they reach the gate.

- R1 gives the most careful “what should not be misdiagnosed” list (sign flips with a compensating accumulator, non-strict comparators, one-argument `MinimalPolynomial`) and a discussion of the discriminant/index relation $\text{Disc}(P) = \text{Disc}(K)[\mathcal{O}_K : \mathbb{Z}[\alpha]]^2$. Its proposed regression `Factorc[Sqrt[t + t^2]]` does *not* exhibit the predicted failure: the kernel returns `Sqrt[t (1 + t)]`, which is value preserving (H02). The mechanism it describes is real, but the concrete probe was mis-chosen; `Sqrt[-x - y]` (R3) and the numeric `Sqrt[-(Sqrt[2] + (-1)^(1/3))]` (R1’s own algebraic example, never run) are the probes that fail.
- R2 and R3 correctly hedge that the wrapper keeps a leading minus sign outside the marker, so `Strad[-Sqrt[3 + 2 Sqrt[2]]]` is *not* a wrapper-level counterexample (C01 confirms), whereas the direct engine call is (D01).
- Only R3 notes that `history[[2, -1]]` in the stopping rule is the largest finite degree seen so far rather than a fixed baseline.

4.4 What the reviews could not know

Because none of them executed the program, several important facts were invisible to all three:

- The mis-typed comparator does not merely fail to sort: with Wolfram’s merge sort it *reverses* the list (S01, N01). Consequently the ratio-pruning loop (lines 184–189) is dead code, the prefix (lines 190–194) selects the *largest* primes, and the stale-length defect U6 can never fire in the original. Repairing the comparator alone activates U6 (N02).
- The acceptance test is not merely heuristic in principle: in the kernel, `PossibleZeroQ` reports “*Unable to decide whether ... is equal to zero. Assuming it is.*” on most successful runs (A03, A05, A07, A08, ...; 14/40 of family F1, 38/40 of F4). The program’s successes rest on that “assume zero” fallback.
- The factorizer changes the value of purely numeric input end to end: `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]` returns the negative of the input (J01), with no message.
- `Strad` on a list returns a `Power`, not a list (K14); on a `Root` object it returns a *nested* radical (B07).
- Loading the file prints a `Syntax::com` warning caused by `Throw[, "solution"]` at lines 320.
- Several “successful” denestings are much larger than the input and contain denestable radicals (D07, L04, A30/A31, Q01–Q08).
- Performance is not the practical problem the reviews feared for inputs of this size: every probe finished in under 2.5s except a 10th root (20s), and the worst batch materialized 63 multipliers.

4.5 The reviews’ own test suites, executed

Each review ships Wolfram tests it did not run. We ran them (Section 5.1). The outcomes are exactly as the reviews predicted where they made predictions: on the original program, every preservation contract for U1, U2, U5, U8, U10, U11 fails and every positive control passes. R2’s suite:

4 of 11 original-program tests pass (loading test, worked example, rationalizer, all-level control). The guarded one-trial components of all three reviews, executed for the first time here, behave as designed on the negative and complex targets, reject a zero multiplier and root order one, and return structured failures; details are in Section 5.5. None of them is a full denester, as their authors say.

5 Experiments

5.1 Setup

All runs used `wolframscript` with *15.0.1 for Microsoft Windows (64-bit) (July 2, 2026)* on the machine holding the repository; the license allows one kernel at a time, so every script ran in a fresh kernel in sequence. A small harness (`runner.wl`, Appendix C) loads the program, evaluates each probe under `TimeConstrained` (120s unless stated), captures every message that is switched on through `Internal`HandlerBlock`, and records:

- *Equal*: exact equality, `RootReduce[result - expected] === 0`, between the result and the independently known value of the input (for probes without a known closed form, 40-digit numeric agreement with the input is shown instead); **NO** therefore means a wrong value;
- *Depth*: rdepth before and after;
- wall-clock time and the list of messages.

The original program prints its diagnostics; **Strad** suppresses them but not messages, and both are visible in the logs. Tables are generated from the recorded runs by `export_tables.wl`.

5.2 The original program

Classical identities (Table 4). The program denests all 26 identities in group A that admit a denesting, including cube, fourth, fifth, sixth and seventh roots, sums and products of nested radicals, complex radicands, and Ramanujan’s $\sqrt[3]{\sqrt[3]{2} - 1}$, each in well under a second. A13, $\sqrt[3]{\sqrt[3]{5} - \sqrt[3]{4}}$, is correctly left alone: the kernel shows that its minimal polynomial has degree 27, so it does not lie in the degree-9 field $\mathbb{Q}(\sqrt[3]{2}, \sqrt[3]{5})$, and the closed form $(\sqrt[3]{2} + \sqrt[3]{20} - \sqrt[3]{25})/3$ that we first expected for it is not an identity (`RootReduce` shows that the cube of that expression is a cubic irrational). Two observations temper the success:

- 19 of the 31 rows carry the message `PossibleZeroQ::ztest1` (“Unable to decide ... Assuming it is”) together with `N::meprec`. The candidate filter did not *decide* equality; it assumed it.
- A27 $(3+2\sqrt{2})^{1/4} \rightarrow \sqrt{1+\sqrt{2}}$ is equal and smaller but not shallower; A30/A31, the depth-three example of Borodin, Fagin, Hopcroft and Tompa, comes back as

$$\frac{\sqrt{319 - 58\sqrt{29}} - 10\sqrt{11 - 2\sqrt{29}} + \sqrt{5}(3\sqrt{29} - 16)}{-5 + \sqrt{29} - \sqrt{55 - 10\sqrt{29}}},$$

which is equal to the input and one level shallower, but far from the answer $\sqrt{5} + \sqrt{11 - 2\sqrt{29}}$ (found by the corrected version, Table 16).

Table 4: Original program on classical denesting identities. “Equal” is exact equality (RootReduce) with the expected value; where an expected value was not supplied, numeric agreement with the input is shown. Depth is the syntactic radical depth before and after.

ID	Input	Output	Equal	Depth	Time (s)
A01	Strad[Sqrt[3 + 2*Sqrt[2]]]	1 + Sqrt[2]	yes	2→1	0.146
A02	Strad[Sqrt[5 + 2*Sqrt[6]]]	Sqrt[2] + Sqrt[3]	yes	2→1	0.040
A03	Strad[Sqrt[2 + Sqrt[3]]]	(1 + Sqrt[3])/Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.072
A04	Strad[Sqrt[11 + 6*Sqrt[2]]]	3 + Sqrt[2]	yes	2→1	0.016
A05	Strad[Sqrt[3 + Sqrt[5]]]	(1 + Sqrt[5])/Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.030
A06	Strad[Sqrt[7 + 2*Sqrt[10]]]	Sqrt[2] + Sqrt[5]	yes	2→1	0.030
A07	Strad[Sqrt[6 + 2*Sqrt[2] + 2*Sqrt[3] + 2*Sqrt[6]]]	1 + Sqrt[2] + Sqrt[3] <i>PossibleZeroQ::ztest1, General::stop</i>	yes	2→1	0.054
A08	Strad[Sqrt[12 + 2*Sqrt[6] + 2*Sqrt[14] + 2*Sqrt[21]]]	Sqrt[2] + Sqrt[3] + Sqrt[7] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.096
A09	Strad[(Sqrt[5] - 2)^(1/3)]	(-1 + Sqrt[5])/2 <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.027
A10	Strad[(2 + Sqrt[5])^(1/3)]	(1 + Sqrt[5])/2 <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.019
A11	Strad[(10 + 6*Sqrt[3])^(1/3)]	1 + Sqrt[3] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.029
A12	Strad[(2^(1/3) - 1)^(1/3)]	(1 - 2^(1/3) + 2^(2/3))/3^(2/3) <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.052
A13	Strad[(5^(1/3) - 4^(1/3))^(1/3)]	(-2^(2/3) + 5^(1/3))^(1/3)	yes	2→2	1.063
A14	Strad[((11 + 5*Sqrt[5])/2)^(1/5)]	(1 + Sqrt[5])/2 <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.031
A15	Strad[Sqrt[1 + 2*2^(1/3) + 2^(2/3)]]	1 + 2^(1/3)	yes	2→1	0.004
A16	Strad[(17 + 12*Sqrt[2])^(1/4)]	1 + Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.018
A17	Strad[(99 + 70*Sqrt[2])^(1/6)]	1 + Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.019
A18	Strad[(3 + 2*Sqrt[2])^(3/2)]	7 + 5*Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.026
A19	Strad[(3 + 2*Sqrt[2])^(-2^(-1))]	-1 + Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.018
A20	Strad[1/Sqrt[3 + 2*Sqrt[2]]]	-1 + Sqrt[2]	yes	2→1	0.009
A21	Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]	1 + 2*Sqrt[2] + Sqrt[3]	yes	2→1	0.030
A22	Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[5 + 2*Sqrt[6]]]	2 + Sqrt[2] + Sqrt[3] + Sqrt[6] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.046
A23	Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]	Sqrt[6]	yes	2→1	0.048
A24	Strad[Sqrt[-1 + 2*I*Sqrt[2]]]	1 + I*Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.026

ID	Input	Output	Equal	Depth	Time (s)
A25	Strad[Sqrt[2 + 2*I*Sqrt[3]]]	I + Sqrt[3] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.046
A26	Strad[Sqrt[-3 - 2*Sqrt[2]]]	I*(1 + Sqrt[2])	yes	2→1	0.009
A27	Strad[(3 + 2*Sqrt[2])^(1/4)]	Sqrt[1 + Sqrt[2]] <i>PossibleZeroQ::ztest1</i>	yes	2→2	0.030
A28	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], True]	Sqrt[5 + 2*Sqrt[2]]	yes	3→2	0.090
A29	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]	Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]	yes	3→3	0.099
A30	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]	(Sqrt[319 - 58*Sqrt[29]] - 10*Sqrt[11 - 2*Sqrt[29]] + Sqrt[5]*(-16 + 3*Sqrt[29]))/(-5 + Sqrt[29] - Sqrt[55 - 10*Sqrt[29]]) <i>PossibleZeroQ::ztest1</i>	yes	3→2	0.598
A31	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]]]	(Sqrt[319 - 58*Sqrt[29]] - 10*Sqrt[11 - 2*Sqrt[29]] + Sqrt[5]*(-16 + 3*Sqrt[29]))/(-5 + Sqrt[29] - Sqrt[55 - 10*Sqrt[29]]) <i>PossibleZeroQ::ztest1</i>	yes	3→2	0.118

Non-denestable and trivial inputs (Table 5). Values are preserved. B01 shows the final Simplify rewriting a non-denestable input into a different, not simpler, form; B07 shows a Root object turned into a *nested* radical (U20).

Table 5: Original program on non-denestable and trivial inputs.

ID	Input	Output	Equal	Depth	Time (s)
B01	Strad[Sqrt[2 + Sqrt[2]]]	2^(1/4)*Sqrt[1 + Sqrt[2]]	yes	2→2	0.031
B02	Strad[Sqrt[1 + Sqrt[2]]]	Sqrt[1 + Sqrt[2]]	yes	2→2	0.022
B03	Strad[(1 + Sqrt[2])^(1/3)]	(1 + Sqrt[2])^(1/3)	yes	2→2	0.085
B04	Strad[Sqrt[2]]	Sqrt[2]	yes	1→1	0.002
B05	Strad[2]	2	yes	0→0	0.000
B06	Strad[Sqrt[1 + I]]	Sqrt[1 + I]	yes	1→1	0.002
B07	Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]	Sqrt[5 + 2*Sqrt[6]]	yes	0→2	0.006
B08	Strad[Sqrt[Sqrt[2] + Sqrt[3]]]	Sqrt[Sqrt[2] + Sqrt[3]]	yes	2→2	0.074
B09	Strad[(2 + 2*I*Sqrt[3])^(1/3)]	(2 + (2*I)*Sqrt[3])^(1/3)	yes	2→2	0.261

Branch counterexamples through the wrapper (Table 6). C02 and C03 are the wrong values predicted by all three reviews: the program returns $5 - i\sqrt{2}$ for $(-1 + 2i\sqrt{2})^{3/2} = -5 + i\sqrt{2}$, and $(\sqrt{2} - i)(\sqrt{3} - i)$ for $\sqrt{-1 + 2i\sqrt{2}}\sqrt{-2 + 2i\sqrt{3}} = (1 + i\sqrt{2})(1 + i\sqrt{3})$. C01, C04, C07, C08 are right because their values happen to lie in the principal sector of Proposition 3.3 ($\text{Arg} = -\pi/2$ is still inside for $q = 2$).

Table 6: Original program on branch counterexamples (wrap-per level). “Equal” compares with the true value of the input.

ID	Input	Output	Equal	Depth	Time (s)
C01	Strad[-Sqrt[3 + 2*Sqrt[2]]]	-1 - Sqrt[2]	yes	2→1	0.016
C02	Strad[(-1 + 2*I*Sqrt[2])^(3/2)]	5 - I*Sqrt[2] <i>PossibleZeroQ::ztest1</i>	NO	2→1	0.396
C03	Strad[Sqrt[-1 + 2*I*Sqrt[2]]* Sqrt[-2 + 2*I*Sqrt[3]]]	(-I + Sqrt[2])*(-I + Sqrt[3]) <i>PossibleZeroQ::ztest1</i>	NO	2→1	0.128
C04	Strad[(-3 - 2*Sqrt[2])^(3/2)]	(-I)*(7 + 5*Sqrt[2])	yes	2→1	0.013
C05	Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	-Sqrt[2*(7 + 5*Sqrt[2])]	yes	2→2	0.065
C06	Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	-3 - 2*Sqrt[2]	yes	1→1	0.009
C07	Strad[(-1 + 2*I*Sqrt[2])^(5/2)]	1 - (11*I)*Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.034
C08	Strad[(-5 - 2*Sqrt[6])^(3/2)]	(-I)*(11*Sqrt[2] + 9*Sqrt[3]) <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.045

The engine called directly (Table 7). D01, D04, D05 reproduce the branch loss at the engine level. D02 (multiplier 0) prints “success” and returns the unevaluated `{ }[[1]]`; D10 (an inexact multiplier) returns `Outer[][[1]]`; D11 (π) is harmless. D06 versus D07 is R2’s equal-degree gate: with $\{1, m\}$ the trial m is skipped because its degree equals the best degree, with $\{m\}$ alone it is tried and “succeeds” with an uglier expression of the same depth (U9, U12). D08 shows that **Extension** → **Automatic** copes with a nested radicand but that single-level mode cannot find $\sqrt{3 + 2\sqrt{3 + 2\sqrt{2}}} = \sqrt{5 + 2\sqrt{2}}$; it tries 12 multipliers and gives up (compare A28). D12 shows that a “forced” list does not prevent generated batches. D14 shows that the imaginary multiplier i works.

Table 7: Original core `denest11` called directly with forced multipliers.

ID	Input	Output	Equal	Depth	Time (s)
D01	denest11[-Sqrt[3 + 2*Sqrt[2]], {1}]	1 + Sqrt[2]	NO	2→1	0.005
D02	denest11[Sqrt[3 + 2*Sqrt[2]], {0}]	{ }[[1]] <i>Power::infy, Part::partw</i>	NO	2→0	0.089
D03	denest11[Sqrt[3 + 2*Sqrt[2]], {}]	Sqrt[3 + 2*Sqrt[2]]	yes	2→2	0.001
D04	denest11[(-1 + 2*I*Sqrt[2])^(3/2), {1}]	5 - I*Sqrt[2]	NO	2→1	0.010
D05	denest11[Sqrt[-1 + 2*I*Sqrt[2]]* Sqrt[-2 + 2*I*Sqrt[3]], {1}]	(-I + Sqrt[2])*(-I + Sqrt[3]) <i>PossibleZeroQ::ztest1</i>	NO	2→1	0.026
D06	denest11[Sqrt[5 + 2*Sqrt[6]], {1, 6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	Sqrt[5 + 2*Sqrt[6]]	yes	2→2	0.012

ID	Input	Output	Equal	Depth	Time (s)
D07	denest11[Sqrt[5 + 2*Sqrt[6]], {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	(1 + Sqrt[2] + Sqrt[3])/Sqrt[2*(3 - Sqrt[2] + Sqrt[3] - Sqrt[6])] <i>PossibleZeroQ::ztest1</i>	yes	2→2	0.056
D08	denest11[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]	Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]	yes	3→3	0.100
D09	denest11[2 + Sqrt[3]]	2 + Sqrt[3]	yes	1→1	0.000
D10	denest11[Sqrt[3 + 2*Sqrt[2]], {2.}]	Outer[][[1]] <i>MinimalPolynomial::nalg, Outer::heads, Outer::argm, Part::partw</i>	NO	2→0	0.238
D11	denest11[Sqrt[3 + 2*Sqrt[2]], {Pi}]	Sqrt[3 + 2*Sqrt[2]] <i>MinimalPolynomial::nalg</i>	yes	2→2	0.004
D12	denest11[Sqrt[5 + 2*Sqrt[6]], {1, Sqrt[5]}]	Sqrt[2] + Sqrt[3]	yes	2→1	0.014
D13	denest11[Sqrt[3 + 2*Sqrt[2]], {-1}]	Sqrt[3 + 2*Sqrt[2]]	yes	2→2	0.005
D14	denest11[Sqrt[3 + 2*Sqrt[2]], {I}]	1 + Sqrt[2] <i>PossibleZeroQ::ztest1</i>	yes	2→1	0.032

Symbolic input (Table 8). $\sqrt{z^2} \mapsto z$ and $\sqrt{-x-y} \mapsto i\sqrt{x+y}$ are the branch-unsafe rewrites of U2 reaching the public interface. E07 is R2's marker collision: `Strad[f[1], False, Identity]` returns 1. E08 shows what a user function named `f` wrapped around a radical provokes: a cascade of seven different messages.

Table 8: Original program on symbolic input.

ID	Input	Output	Equal	Depth	Time (s)
E01	Strad[Sqrt[z^2]]	z	—	1→0	0.001
E02	Strad[Sqrt[t + t^2]]	Sqrt[t*(1 + t)]	—	1→1	0.002
E03	Strad[Sqrt[-x - y]]	I*Sqrt[x + y]	—	1→1	0.004
E04	Strad[Sqrt[a + b]]	Sqrt[a + b]	—	1→1	0.002
E05	Strad[x]	x	—	0→0	0.000
E06	Strad[Sqrt[x]*Sqrt[3 + 2*Sqrt[2]]]	(1 + Sqrt[2])*Sqrt[x]	—	2→1	0.010
E07	Strad[f[1], False, Identity]	1	—	0→0	0.001
E08	Strad[f[Sqrt[3 + 2*Sqrt[2]]]]	f[Sqrt[3 + 2*Sqrt[2]]] <i>MinimalPolynomial::nalg, General::stop, Discriminant::poly2, FactorInteger::exact, Part::partd, Union::heads, Part::pkspec1</i>	—	2→2	0.363

Helpers (Tables 9 and 10). `Factorc` preserves the value on 18 of 20 numeric probes; H05 is the numeric counterexample of Proposition 3.5(2): the result $i\sqrt{(-1)^{1/3} + \sqrt{2}}$ is the negative of the input. Note that `Factorc[Sqrt[t + t^2]]` (H02) returns `Sqrt[t (1 + t)]`, so R1's probe

does not trigger the defect. `RationalizeDenominator` is correct on every numeric denominator (R01–R10); R05 confirms R2’s prediction that a numerator named `x` is erased (result 0), and R11 shows the helper emitting three messages on a symbolic denominator. S01 shows the comparator reversing a list; S02 the out-of-range slice; S04–S06 that `PossibleZeroQ` correctly rejects three very close non-zero differences, so [U3](#) is a matter of unreliable *acceptance*, not of easily demonstrable false positives.

Table 9: `Factorc` on symbolic and numeric inputs. “Equal” is exact equality with the input.

ID	Input	Output	Equal	Depth	Time (s)
H01	<code>Factorc[Sqrt[z^2]]</code>	<code>z</code>	–	0→0	0.000
H02	<code>Factorc[Sqrt[t + t^2]]</code>	<code>Sqrt[t*(1 + t)]</code>	–	1→1	0.000
H03	<code>Factorc[Sqrt[-x - y]]</code>	<code>I*Sqrt[x + y]</code>	–	1→1	0.001
H04	<code>Factorc[Sqrt[-2 - 2*Sqrt[2]]]</code>	<code>I*Sqrt[2*(1 + Sqrt[2])]</code>	yes	2→2	0.001
H05	<code>Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]]</code>	<code>I*Sqrt[(-1)^(1/3) + Sqrt[2]]</code>	NO	2→2	0.001
H06	<code>Factorc[Sqrt[(1 + I)*(3 + 2*Sqrt[2])]]]</code>	<code>Sqrt[(1 + I)*(3 + 2*Sqrt[2])]</code>	yes	2→2	0.001
H07	<code>Factorc[Sqrt[-1 - I]]</code>	<code>Sqrt[-1 - I]</code>	yes	1→1	0.000
H08	<code>Factorc[Sqrt[-2 - 2*I*Sqrt[3]]]</code>	<code>Sqrt[2*(-1 - I*Sqrt[3])]</code>	yes	2→2	0.001
H09	<code>Factorc[Sqrt[(-1 - I*Sqrt[3])*(1 + Sqrt[2])]]]</code>	<code>Sqrt[(-I)*(1 + Sqrt[2])*(-I + Sqrt[3])]</code>	yes	2→2	0.001
H10	<code>Factorc[(-3 - 2*Sqrt[2])^(1/3)]</code>	<code>(-3 - 2*Sqrt[2])^(1/3)</code>	yes	2→2	0.000
H11	<code>Factorc[((-1 + I)*(3 + 2*Sqrt[2]))^(1/3)]</code>	<code>((-1 + I)*(3 + 2*Sqrt[2]))^(1/3)</code>	yes	2→2	0.001
H12	<code>Factorc[Sqrt[2] + Sqrt[3]]</code>	<code>Sqrt[2] + Sqrt[3]</code>	yes	1→1	0.001
H13	<code>Factorc[(1 + Sqrt[2])^2]</code>	<code>(1 + Sqrt[2])^2</code>	yes	1→1	0.000
H14	<code>Factorc[Sqrt[8 + 4*Sqrt[3]]]</code>	<code>2*Sqrt[2 + Sqrt[3]]</code>	yes	2→2	0.001
H15	<code>Factorc[Sqrt[-8 - 4*Sqrt[3]]]</code>	<code>(2*I)*Sqrt[2 + Sqrt[3]]</code>	yes	2→2	0.001
H16	<code>Factorc[Sqrt[(-1)^(1/3)*(3 + 2*Sqrt[2])]]]</code>	<code>(-1)^(1/6)*Sqrt[3 + 2*Sqrt[2]]</code>	yes	2→2	0.001
H17	<code>Factorc[Sqrt[(-1)^(2/3)*(3 + 2*Sqrt[2])]]]</code>	<code>(-1)^(1/3)*Sqrt[3 + 2*Sqrt[2]]</code>	yes	2→2	0.001
H18	<code>Factorc[Sqrt[x^4]]</code>	<code>x^2</code>	–	0→0	0.000
H19	<code>Factorc[(x^2)^(1/3)]</code>	<code>x^(2/3)</code>	–	1→1	0.000
H20	<code>Factorc[Sqrt[4*x^2 + 4*x^3]]</code>	<code>2*Sqrt[x^2*(1 + x)]</code>	–	1→1	0.001
H21	<code>Factorc[Sqrt[-3 - 2*Sqrt[2]]]</code>	<code>I*Sqrt[3 + 2*Sqrt[2]]</code>	yes	2→2	0.001
H22	<code>Factorc[Sqrt[(-1 + I*Sqrt[3])*(-1 + I*Sqrt[3])]]]</code>	<code>1 - I*Sqrt[3]</code>	yes	1→1	0.000
H23	<code>Factorc[Sqrt[-(1 + Sqrt[2]) - (1 + Sqrt[2])*I*Sqrt[3]]]</code>	<code>Sqrt[(-I)*(1 + Sqrt[2])*(-I + Sqrt[3])]</code>	yes	2→2	0.001

Table 10: Helper-level probes: `RationalizeDenominator`,
marker collisions, nesting helpers, comparator, slice,
`PossibleZeroQ`.

ID	Input	Output	Equal	Depth	Time (s)
R01	<code>RationalizeDenominator[1/(1 + Sqrt[2])]</code>	<code>-1 + Sqrt[2]</code>	yes	1→1	0.000
R02	<code>RationalizeDenominator[1/(Sqrt[2] + Sqrt[3])]</code>	<code>-Sqrt[2] + Sqrt[3]</code>	yes	1→1	0.002
R03	<code>RationalizeDenominator[1/(1 + I*Sqrt[2])]</code>	<code>(1 - I*Sqrt[2])/3</code>	yes	1→1	0.001
R04	<code>RationalizeDenominator[(1 + Sqrt[2])/(Sqrt[2] - Sqrt[3])]</code>	<code>-2 - Sqrt[2] - Sqrt[3] - Sqrt[6]</code>	yes	1→1	0.002
R05	<code>RationalizeDenominator[x/(1 + Sqrt[2])]</code>	0	yes	0→0	0.000
R06	<code>RationalizeDenominator[1/2 + 1/Sqrt[2]]</code>	<code>1/2 + 1/Sqrt[2]</code>	yes	1→1	0.000
R07	<code>RationalizeDenominator[1/Sqrt[2]]</code>	<code>1/Sqrt[2]</code>	yes	1→1	0.000
R08	<code>RationalizeDenominator[1/(1 + 2^(1/3))]</code>	<code>(1 - 2^(1/3) + 2^(2/3))/3</code>	yes	1→1	0.001
R09	<code>RationalizeDenominator[1/Sqrt[1 + Sqrt[2]]]</code>	<code>-Sqrt[1 + Sqrt[2]] + Sqrt[2*(1 + Sqrt[2])]</code>	yes	2→2	0.001
R10	<code>RationalizeDenominator[0]</code>	0	yes	0→0	0.000
R11	<code>RationalizeDenominator[1/x]</code>	<code>-(PolynomialQuotient[0, 0, 0]/MinimalPolynomial[x][0])</code> <i>MinimalPolynomial::nalg, General::stop, PolynomialQuotient::ivar</i>	—	0→0	0.008
M01	<code>DenestRadicals3[f[1], False, Identity]</code>	1	—	0→0	0.000
M02	<code>DenestRadicals3[g[1], False, Identity]</code>	<code>g[1]</code>	—	0→0	0.000
M03	<code>DenestRadicals3[f[Sqrt[3 + 2*Sqrt[2]]], False, Identity]</code>	<code>f[Sqrt[3 + 2*Sqrt[2]]]</code> <i>General::newsym</i>	—	2→2	0.007
N01	<code>maxnestdepth[Sqrt[3 + 2*Sqrt[2]], MatchQ[#1, _Rational] &]</code>	<code>{{Sqrt[3 + 2*Sqrt[2]], 2}}</code>	—	2→2	0.000
N02	<code>maxnestdepth[(1 + 2^(1/3))^3, MatchQ[#1, _Rational] &]</code>	<code>{{2^(1/3), 1}}</code>	—	1→1	0.000
N03	<code>LCM @@ (maxnestdepth[(1 + 2^(1/3))^3, MatchQ[#1, _Rational] &] /. {(base_)^(exp_), _} :> Denominator[exp])</code>	3	yes	0→0	0.000
N04	<code>maxnestdepth[Sqrt[1 + 2^(1/3)]*(1 + Sqrt[2])^(1/3), MatchQ[#1, _Rational] &]</code>	<code>{{Sqrt[1 + 2^(1/3)], 2}, {(1 + Sqrt[2])^(1/3), 2}}</code>	—	2→2	0.000

ID	Input	Output	Equal	Depth	Time (s)
N05	nestdepthsort[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], MatchQ[#1, _ _Rational] &]	{{{Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], 3}}, {{3 + 2*Sqrt[3 + 2*Sqrt[2]]], 2}, {2*Sqrt[3 + 2*Sqrt[2]]], 2}, {Sqrt[3 + 2*Sqrt[2]]], 2}}, {{3 + 2*Sqrt[2]], 1}, {2*Sqrt[2]], 1}, {Sqrt[2]], 1}}, {{3, 0}, {2, 0}, {3, 0}, {2, 0}, {2, 0}, {1/2, 0}, {1/2, 0}, {1/2, 0}}}}	—	3→3	0.000
N06	maxnestdepth[Sqrt[3 + 2*Sqrt[2]] + (5 + 2*Sqrt[6])^(1/3), MatchQ[#1, _ _Rational] &]	{{Sqrt[3 + 2*Sqrt[2]], 2}, {(5 + 2*Sqrt[6])^(1/3), 2}}	—	2→2	0.000
S01	Sort[{{2, 1}, {3, 1}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {2003, 1}}, #1[[1]] <= #1[[2]] &]	{{2003, 1}, {13, 1}, {11, 1}, {7, 1}, {5, 1}, {3, 1}, {2, 1}}	—	0→0	0.000
S02	Module[{p = {2, 3, 5, 7, 11, 0}}, DeleteCases[p, 0][[1 ;; Min[10, Length[p]]]]]	{2, 3, 5, 7, 11}[[1 ;; 6]] <i>Part::take</i>	—	0→0	0.000
S03	ComplexitySort[{6, Sqrt[2], 2, 3, 1 + Sqrt[2], 10, -3}]	{2, 3, -3, 6, 10, Sqrt[2], 1 + Sqrt[2]}	—	1→1	0.000
S04	PossibleZeroQ[Sqrt[2] - FromContinuedFraction[ContinuedFraction[Sqrt[2], 400]]]	False	—	0→0	0.000
S05	PossibleZeroQ[Sqrt[2] - FromContinuedFraction[ContinuedFraction[Sqrt[2], 400]], Method -> "ExactAlgebraics"]	False <i>N::meprec</i>	—	0→0	0.001
S06	PossibleZeroQ[Sqrt[2] - Root[#1^2 - 2 + 10^(-100) & , 2]]	False	—	0→0	0.001
S07	DeleteDuplicates[{1 + Sqrt[2], Sqrt[3 + 2*Sqrt[2]], 1 + Sqrt[2]}]	{1 + Sqrt[2], Sqrt[3 + 2*Sqrt[2]]}	—	2→2	0.000

Wrapper-level numeric branch errors and odd inputs (Table 11). J01 and J02 are the most important new findings: a purely numeric input, $\sqrt{-(\sqrt{2} + (-1)^{1/3})}$, alone or inside a sum, is returned with the wrong sign, silently. J06 is the cube-root analogue. K14 shows that a *list* of two radicals is turned into $(3 + 2\sqrt{2})^{(\sqrt{2}+\sqrt{3})/2}$. K04 shows that the ugly BFHT output is a fixed point of the program. K12: inexact input is returned as a machine number.

Table 11: Original program: wrapper-level numeric branch errors caused by **Factorc** (J) and miscellaneous inputs (K).

ID	Input	Output	Equal	Depth	Time (s)
J01	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]	I*Sqrt[(-1)^(1/3) + Sqrt[2]]	NO	2→2	0.097

ID	Input	Output	Equal	Depth	Time (s)
J02	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3)) + 1]]	$1 + I \cdot \text{Sqrt}[(-1)^{(1/3)} + \text{Sqrt}[2]]$	NO	2→2	0.101
J03	Factorc[Sqrt[(-(1 + Sqrt[2]))*((1 + I*Sqrt[3])/2)]]	$\text{Sqrt}[(-1/2*I)*(1 + \text{Sqrt}[2])*(-I + \text{Sqrt}[3])]$	yes	2→2	0.004
J04	Strad[Sqrt[(-(1 + Sqrt[2]))*((1 + I*Sqrt[3])/2)]]	$\text{Sqrt}[(-1/2*I)*(1 + \text{Sqrt}[2])*(-I + \text{Sqrt}[3])]$	yes	2→2	0.313
J05	Strad[Sqrt[-3 - 2*Sqrt[2] - I*(3 + 2*Sqrt[2])*Sqrt[3]]]	$((2 + \text{Sqrt}[2])*(1 - I \cdot \text{Sqrt}[3]))/2$	yes	1→1	0.026
J06	Strad[(-(Sqrt[2] + (-1)^(1/3)))^(1/3)]	$(-1)^{(1/3)}*((-1)^{(1/3)} + \text{Sqrt}[2])^{(1/3)}$	NO	2→2	0.792
J07	Strad[Sqrt[-Sqrt[2] - I*Sqrt[3]]]	$\text{Sqrt}[-\text{Sqrt}[2] - I \cdot \text{Sqrt}[3]]$	yes	2→2	0.109
J08	Strad[Sqrt[-1 - I*Sqrt[3] - Sqrt[2] - I*Sqrt[6]]]	$\text{Sqrt}[(-I)*(1 + \text{Sqrt}[2])*(-I + \text{Sqrt}[3])]$	yes	2→2	0.450
K01	Strad[Sqrt[28^(1/3) - 3]]	$(-1 - 2^{(2/3)}*7^{(1/3)} + 2^{(1/3)}*7^{(2/3)})/3$	yes	1→1	0.018
K02	Strad[(1 + 2^(1/3))^3]	$(1 + 2^{(1/3)})^3$	yes	1→1	0.002
K03	denest11[(1 + 2^(1/3))^3]	$3*(1 + 2^{(1/3)} + 2^{(2/3)})$	yes	1→1	0.006
K04	Strad[Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]]	$(\text{Sqrt}[319 - 58 \cdot \text{Sqrt}[29]] - 10 \cdot \text{Sqrt}[11 - 2 \cdot \text{Sqrt}[29]] + \text{Sqrt}[5] \cdot (-16 + 3 \cdot \text{Sqrt}[29]))/(-5 + \text{Sqrt}[29] - \text{Sqrt}[55 - 10 \cdot \text{Sqrt}[29]])$ <i>PossibleZeroQ::ztest1</i>	yes	2→2	0.370
K05	Strad[Sqrt[2 + Sqrt[3]]*Sqrt[2 - Sqrt[3]]]	1	yes	0→0	0.001
K06	Strad[Sqrt[Sqrt[2] - 1]*Sqrt[Sqrt[2] + 1]]	1	yes	0→0	0.000
K07	Strad[Sqrt[3 + 2*Sqrt[2]]^(1/3)]	$(1 + \text{Sqrt}[2])^{(1/3)}$	yes	2→2	0.010
K08	Strad[(3 + 2*Sqrt[2])^(1/6)]	$(1 + \text{Sqrt}[2])^{(1/3)}$	yes	2→2	0.012
K09	Strad[Sqrt[5 + 2*Sqrt[6]]/Sqrt[3 + 2*Sqrt[2]]]	$2 - \text{Sqrt}[2] - \text{Sqrt}[3] + \text{Sqrt}[6]$	yes	1→1	0.024
K10	Strad[Exp[I*(Pi/7)]*Sqrt[3 + 2*Sqrt[2]]]	$(-1)^{(1/7)}*(1 + \text{Sqrt}[2])$	yes	1→1	0.010
K11	Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]	$1 + \text{Sqrt}[2] + \text{Pi}$	yes	1→1	0.010
K12	Strad[Sqrt[3. + 2*Sqrt[2]]]	2.414213562373095	yes	0→0	0.000
K13	Strad[Sqrt[2 + Sqrt[3]], "yes"]	$(1 + \text{Sqrt}[3])/ \text{Sqrt}[2]$	yes	1→1	0.013
K14	Strad[{Sqrt[3 + 2*Sqrt[2]], Sqrt[5 + 2*Sqrt[6]]}]	$(3 + 2 \cdot \text{Sqrt}[2])^{((\text{Sqrt}[2] + \text{Sqrt}[3])/2)}$	yes	1→1	0.019
K15	Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]	True	—	0→0	0.002
K16	Strad[Sqrt[9 + 4*Sqrt[5]]]	$2 + \text{Sqrt}[5]$	yes	1→1	0.009
K17	Strad[Sqrt[(3 + 2*Sqrt[2])/(5 + 2*Sqrt[6])]]	$-2 - \text{Sqrt}[2] + \text{Sqrt}[3] + \text{Sqrt}[6]$	yes	1→1	0.024
K18	Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[x]]	$(1 + \text{Sqrt}[2]) \cdot \text{Sqrt}[x]$	yes	1→1	0.008
K19	Strad[Sqrt[Sqrt[2] + 1]^3]	$(1 + \text{Sqrt}[2])^{(3/2)}$	yes	2→2	0.028
K20	Strad[Sqrt[2 + Sqrt[3]]*Sqrt[3 + 2*Sqrt[2]]]	$((2 + \text{Sqrt}[2])*(1 + \text{Sqrt}[3]))/2$	yes	1→1	0.017

Higher-order roots, PossibleZeroQ, prime processing (Table 12). L04, a four-term square root, is denested to a correct but enormous quotient (rationalizing its denominator and simplifying gives $\sqrt{2} + \sqrt{3} + \sqrt{5} + \sqrt{7}$; the corrected version does this). L14, a tenth root, takes 20 s and emits `Ceiling::meprec` because `Ceiling[Log[1000]/Log[10]]` does not evaluate (R01–R04 in Table 13). N01–N04 compare the prime processing with the original and the corrected comparator on the same factor lists: the original reverses the list and never prunes; the corrected one sorts, prunes, and then hits the stale-length `Part::take` (U6). N05/N06 materialize the 2^{10} and 10^5 divisor lists that the “cap” permits.

Table 12: Original program: higher-order roots and timing (L), PossibleZeroQ on actual candidate differences (M), prime processing with the original and the corrected comparator (N).

ID	Input	Output	Equal	Depth	Time (s)
L01	<code>Strad[(239 + 169*Sqrt[2])^(1/7)]</code>	<code>1 + Sqrt[2]</code>	yes	1→1	0.008
L02	<code>Strad[(2 + Sqrt[2])^(1/5)]</code>	<code>2^(1/10)*(1 + Sqrt[2])^(1/5)</code>	yes	2→2	0.198
L03	<code>Strad[(1 + Sqrt[2])^(1/7)]</code>	<code>(1 + Sqrt[2])^(1/7)</code>	yes	2→2	0.494
L04	<code>Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]</code>	<code>(6397*Sqrt[2] + 5316*Sqrt[3] + 4090*Sqrt[5] + 3432*Sqrt[7] + 1695*Sqrt[30] + 1423*Sqrt[42] + 1095*Sqrt[70] + 910*Sqrt[105])/((1108 + 467*Sqrt[6] + 359*Sqrt[10] + 299*Sqrt[14] + 302*Sqrt[15] + 252*Sqrt[21] + 194*Sqrt[35] + 81*Sqrt[210])</code>	yes	1→1	0.956
L05	<code>Strad[Expand[(1 + Sqrt[2] + Sqrt[3])^3]^(1/3)]</code>	<code>1 + Sqrt[2] + Sqrt[3]</code>	yes	1→1	0.020
L06	<code>Strad[Expand[(Sqrt[2] + Sqrt[3])^4]^(1/4)]</code>	<code>Sqrt[5 + 2*Sqrt[6]]</code>	yes	2→2	0.013
L07	<code>Strad[Sqrt[1000003 + 2*Sqrt[999983]]]</code>	<code>Sqrt[1000003 + 2*Sqrt[999983]]</code>	yes	2→2	0.215
L08	<code>Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]</code>	<code>Sqrt[2 + Sqrt[3] + Sqrt[5]]</code>	yes	2→2	0.216
L09	<code>Strad[(5 + 2*Sqrt[6])^(1/6)]</code>	<code>(2 + Sqrt[6])^(1/3)/2^(1/6)</code>	yes	2→2	2.256
L10	<code>Strad[(11*Sqrt[2] + 9*Sqrt[3])^(1/3)]</code>	<code>Sqrt[2] + Sqrt[3]</code>	yes	1→1	0.015
L11	<code>Strad[Expand[(2^(1/3) + 3^(1/3))^3]^(1/3)]</code>	<code>2^(1/3) + 3^(1/3)</code>	yes	1→1	0.133
L12	<code>Strad[Sqrt[Expand[(1 + 2^(1/3) + 3^(1/3))^2]]]</code>	<code>1 + 2^(1/3) + 3^(1/3)</code>	yes	1→1	0.006
L13	<code>Strad[Expand[(1 + 2^(1/5))^5]^(1/5)]</code>	<code>1 + 2^(1/5)</code>	yes	1→1	0.023
L14	<code>Strad[Sqrt[5 + 2*Sqrt[6]]^(1/5)]</code>	<code>-(((−1)^(4/5)*(−2 − Sqrt[6])^(1/5))/2^(1/10))</code> <code>Ceiling::meprec</code>	yes	2→2	20.350
M01	<code>PossibleZeroQ[Sqrt[3/2] + 1/Sqrt[2] − Sqrt[2 + Sqrt[3]]]</code>	<code>True</code>	–	0→0	0.000

ID	Input	Output	Equal	Depth	Time (s)
M02	PossibleZeroQ[Sqrt[3/2] + 1/ Sqrt[2] - Sqrt[2 + Sqrt[3]], Method -> "ExactAlgebraics"]	True	—	0→0	0.000
M03	RootReduce[Sqrt[3/2] + 1/Sqrt[2] - Sqrt[2 + Sqrt[3]]]	0	yes	0→0	0.014
M04	PossibleZeroQ[-Sqrt[3/2] - 1/ Sqrt[2] - Sqrt[2 + Sqrt[3]]]	False	—	0→0	0.000
M05	PossibleZeroQ[(1 + I*Sqrt[2])*(1 + I*Sqrt[3]) - Sqrt[Expand[(1 + I*Sqrt[2])^2*(1 + I*Sqrt[3])^2]]]	False	—	0→0	0.000
M06	PossibleZeroQ[(-(1 + I*Sqrt[2]))* (1 + I*Sqrt[3]) - Sqrt[Expand[(1 + I*Sqrt[2])^2*(1 + I*Sqrt[3])^2]]]	True	—	0→0	0.000
N01	primeProc[facs, #1[[1]] <= #1[[2]] & , 2]	{{2, 2003, 11, 7, 5, 3}, {2, 2003, 11, 7, 5, 3}}	—	0→0	0.000
N02	primeProc[facs, #1[[1]] <= #2[[1]] & , 2]	{{2, 3, 5, 7, 11, 2003}, {2, 3, 5, 7, 11}[[1 ;; 6]]} <i>Part::take</i>	—	0→0	0.000
N03	primeProc[{{2, 14}, {3, 2}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {17, 1}, {19, 1}, {23, 1}, {29, 1}, {31, 1}}, #1[[1]] <= #1[[2]] & , 2]	{{2, 31, 29, 23, 19, 17, 13, 11, 7, 5, 3}, {2, 31, 29, 23, 19, 17, 13, 11, 7, 5}}	—	0→0	0.000
N04	primeProc[{{2, 14}, {3, 2}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {17, 1}, {19, 1}, {23, 1}, {29, 1}, {31, 1}}, #1[[1]] <= #2[[1]] & , 2]	{{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31}, {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}}	—	0→0	0.000
N05	Length[Divisors[(2*3*5*7*11*13* 17*19*23*29)^(2 - 1)]]	1024	yes	0→0	0.000
N06	Length[Divisors[(2*3*5*7*11)^(10 - 1)]]	100000	yes	0→0	0.039

Negative bases and output quality (Table 13). Q01, Q02, Q07, Q08: for $(p + q\sqrt{c})^3 < 0$ the principal cube root is $e^{i\pi/3}|p + q\sqrt{c}|$; the program's "solutions" are equal to the input but contain $\sqrt{9 \pm 6\sqrt{2}}$ and similar denestable surds, so the depth does not decrease (U23). Q03–Q06 (fourth roots and positive bases) are fine.

Table 13: Original program: roots of negative bases (Q), the `Ceiling[Log]` issue (R), output quality for a four-term square root (S).

ID	Input	Output	Equal	Depth	Time (s)
Q01	<code>Strad[Expand[(-1 - Sqrt[2])^3]^(1/3)]</code>	$((-1)^{(2/3)} * (1 + \text{Sqrt}[2] - I * \text{Sqrt}[9 + 6 * \text{Sqrt}[2]])) / 2$	yes	2→2	0.021
Q02	<code>Strad[Expand[(1 - Sqrt[2])^3]^(1/3)]</code>	$((-1)^{(2/3)} * (-1 + \text{Sqrt}[2] - I * \text{Sqrt}[9 - 6 * \text{Sqrt}[2]])) / 2$	yes	2→2	0.019
Q03	<code>Strad[Expand[(-1 - Sqrt[2])^4]^(1/4)]</code>	$1 + \text{Sqrt}[2]$	yes	1→1	0.008
Q04	<code>Strad[Expand[(1 - Sqrt[2])^4]^(1/4)]</code>	$-1 + \text{Sqrt}[2]$	yes	1→1	0.013
Q05	<code>Strad[Expand[(2 - 3*Sqrt[3])^4]^(1/4)]</code>	$-2 + 3 * \text{Sqrt}[3]$	yes	1→1	0.015
Q06	<code>Strad[Expand[(3 - Sqrt[2])^3]^(1/3)]</code>	$3 - \text{Sqrt}[2]$	yes	1→1	0.014
Q07	<code>Strad[Expand[(2 - 3*Sqrt[2])^3]^(1/3)]</code>	$-(((-1)^{(2/3)} * (-3 + \text{Sqrt}[2] + I * \text{Sqrt}[33 - 18 * \text{Sqrt}[2]])) / \text{Sqrt}[2])$	yes	2→2	0.023
Q08	<code>Strad[(-7 - 5*Sqrt[2])^(1/3)]</code>	$((-1)^{(2/3)} * (1 + \text{Sqrt}[2] - I * \text{Sqrt}[9 + 6 * \text{Sqrt}[2]])) / 2$	yes	2→2	0.017
Q09	<code>denest11[(7 - 5*Sqrt[2])^(1/3)]</code>	$((-1)^{(2/3)} * (-1 + \text{Sqrt}[2] - I * \text{Sqrt}[9 - 6 * \text{Sqrt}[2]])) / 2$	yes	2→2	0.013
R01	<code>Ceiling[Log[1000]/Log[10]]</code>	<code>Ceiling[Log[1000]/Log[10]]</code> <i>Ceiling::meprec</i>	yes	0→0	0.001
R02	<code>Max[5, Ceiling[Log[1000]/Log[10]]]</code>	5 <i>Ceiling::meprec</i>	yes	0→0	0.001
R03	<code>Min[Max[5, Ceiling[Log[1000]/Log[10]]], 3]</code>	3 <i>Ceiling::meprec</i>	yes	0→0	0.000
R04	<code>{2, 3, 5}[[1 ;; Min[Max[5, Ceiling[Log[1000]/Log[10]]], 3]]</code>	{2, 3, 5} <i>Ceiling::meprec</i>	—	0→0	0.000
R05	<code>Ceiling[Log[1000]/Log[100]]</code>	2	yes	0→0	0.000
R06	<code>Ceiling[Log[1000]/Log[1000]]</code>	1	yes	0→0	0.000
R07	<code>Ceiling[Log[1000]/Log[6]]</code>	4	yes	0→0	0.000
S01	<code>Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]</code>	$(6397 * \text{Sqrt}[2] + 5316 * \text{Sqrt}[3] + 4090 * \text{Sqrt}[5] + 3432 * \text{Sqrt}[7] + 1695 * \text{Sqrt}[30] + 1423 * \text{Sqrt}[42] + 1095 * \text{Sqrt}[70] + 910 * \text{Sqrt}[105]) / (1108 + 467 * \text{Sqrt}[6] + 359 * \text{Sqrt}[10] + 299 * \text{Sqrt}[14] + 302 * \text{Sqrt}[15] + 252 * \text{Sqrt}[21] + 194 * \text{Sqrt}[35] + 81 * \text{Sqrt}[210])$	yes	1→1	2.355

5.3 Randomized families

Ten structured families with a fixed seed (`SeedRandom[20260905]`), $p, q, r \in \{\pm 1, \dots, \pm 7\}$ and c, d square-free in $\{2, \dots, 17\}$, were run through `Strad` of both versions with a 60s limit per input. Each result was classified by exact comparison with the input. Table 14 summarizes; “wrong” means the

returned value differs from the input.

The F3 family is a control: with $a \in [-30, 30]$ random, only 6 of 40 radicands satisfy the square criterion of Proposition 3.6, and both versions denest exactly those 6. The 8 “equal, rewritten” outputs of the original in F3 are the final **Simplify** at work (U20); the corrected version returns such inputs unchanged. The 17 rewritten outputs in F2 and 8 in F7 are the negative-base cases of Table 13.

5.4 The reviews’ regression suites

Table 15 lists the outcome of each review’s own Wolfram tests on the original program. Every “expected to fail” contract fails, every control passes, and no test aborted.

5.5 The reviews’ proposed kernels

The three one-trial components (R1 **VerifiedMultiplierTrial**, R2 **CertifiedMultiplierStep**, R3 **TryDenestCandidate**) were loaded and called on eleven common cases. All three certify the correct branch for the negative target, the complex $3/2$ power, the complex product and the cube-root case with $m = 9$; all three reject a zero multiplier, an inexact multiplier, root order 1 and a symbolic target with a structured **Failure**; all three report “no lower-degree GCD” for $\sqrt{5 + 2\sqrt{6}}$ with $m = 1$ and find $\sqrt{2} + \sqrt{3}$ with $m = 2$, and, because none of them has a degree gate, all three also find it with R2’s equal-degree multiplier (returning the same ugly quotient as the original, D07). R3’s **BestImprovement** policy correctly rejects that quotient as “not an improvement”, but would accept the original’s BFHT output (Table e_{tab}:origA, A31), because its cost tuple puts radical depth before size: (0, 2, 9, 69) beats (0, 3, 4, 28). In other words the components work as their authors said, and they are single steps, not denesters: none contains a multiplier generator, a wrapper, or a search.

5.6 The corrected version

Tables 16–20 repeat the battery with **StradFixed.wl**. Every row is exactly equal to the true value; no row emits a message; **Print** keeps its **Protected** attribute. Beyond fixing every wrong value, the corrected version finds denestings the original does not: the BFHT example (A30/A31), the four-term square root as $\sqrt{2} + \sqrt{3} + \sqrt{5} + \sqrt{7}$ (L04), $(49 + 20\sqrt{6})^{1/4} = \sqrt{2} + \sqrt{3}$ (L06), the cube roots of negative bases (Q01–Q08) and the equal-degree multiplier (D06). It also returns inputs unchanged when it cannot improve them (B01, B08, B09, L02, L03, L09) instead of rewriting them, converts a **Root** input to $\sqrt{2} + \sqrt{3}$ (B07), maps over lists (K14), leaves symbolic hosts alone (E01–E05, E07) while still denesting numeric parts inside them (E06, E08), and rejects invalid multipliers without garbage (D02, D10, D11). The price is a factor of roughly two to three in time on the small inputs, almost entirely spent in **RootReduce** certification.

¹R2’s suite restores the attributes of **Print** itself after loading and then tests the recorded values, so this test measures the suite’s own bookkeeping rather than the defect; R1’s and R3’s versions of the same test fail as they should.

Table 16: Corrected version on the classical identities (compare Table 4).

ID	Input	Output	Equal	Depth	Time (s)
A01	Strad[Sqrt[3 + 2*Sqrt[2]]]	1 + Sqrt[2]	yes	2→1	0.103
A02	Strad[Sqrt[5 + 2*Sqrt[6]]]	Sqrt[2] + Sqrt[3]	yes	2→1	0.055
A03	Strad[Sqrt[2 + Sqrt[3]]]	Sqrt[3/2] + 1/Sqrt[2]	yes	2→1	0.057
A04	Strad[Sqrt[11 + 6*Sqrt[2]]]	3 + Sqrt[2]	yes	2→1	0.030
A05	Strad[Sqrt[3 + Sqrt[5]]]	1/Sqrt[2] + Sqrt[5/2]	yes	2→1	0.060
A06	Strad[Sqrt[7 + 2*Sqrt[10]]]	Sqrt[2] + Sqrt[5]	yes	2→1	0.040
A07	Strad[Sqrt[6 + 2*Sqrt[2] + 2*Sqrt[3] + 2*Sqrt[6]]]	1 + Sqrt[2] + Sqrt[3]	yes	2→1	0.093
A08	Strad[Sqrt[12 + 2*Sqrt[6] + 2*Sqrt[14] + 2*Sqrt[21]]]	Sqrt[2] + Sqrt[3] + Sqrt[7]	yes	2→1	0.231
A09	Strad[(Sqrt[5] - 2)^(1/3)]	(-1 + Sqrt[5])/2	yes	2→1	0.045
A10	Strad[(2 + Sqrt[5])^(1/3)]	(1 + Sqrt[5])/2	yes	2→1	0.046
A11	Strad[(10 + 6*Sqrt[3])^(1/3)]	1 + Sqrt[3]	yes	2→1	0.066
A12	Strad[(2^(1/3) - 1)^(1/3)]	(1 - 2^(1/3) + 2^(2/3))/3^(2/3)	yes	2→1	0.337
A13	Strad[(5^(1/3) - 4^(1/3))^(1/3)]	(-2^(2/3) + 5^(1/3))^(1/3)	yes	2→2	1.503
A14	Strad[((11 + 5*Sqrt[5])/2)^(1/5)]	(1 + Sqrt[5])/2	yes	2→1	0.045
A15	Strad[Sqrt[1 + 2*2^(1/3) + 2^(2/3)]]	1 + 2^(1/3)	yes	2→1	0.019
A16	Strad[(17 + 12*Sqrt[2])^(1/4)]	1 + Sqrt[2]	yes	2→1	0.042
A17	Strad[(99 + 70*Sqrt[2])^(1/6)]	1 + Sqrt[2]	yes	2→1	0.044
A18	Strad[(3 + 2*Sqrt[2])^(3/2)]	7 + 5*Sqrt[2]	yes	2→1	0.027
A19	Strad[(3 + 2*Sqrt[2])^(-2^(-1))]	-1 + Sqrt[2]	yes	2→1	0.049
A20	Strad[1/Sqrt[3 + 2*Sqrt[2]]]	-1 + Sqrt[2]	yes	2→1	0.042
A21	Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]	1 + 2*Sqrt[2] + Sqrt[3]	yes	2→1	0.043
A22	Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[5 + 2*Sqrt[6]]]	2 + Sqrt[2] + Sqrt[3] + Sqrt[6]	yes	2→1	0.094
A23	Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]	Sqrt[6]	yes	2→1	0.058
A24	Strad[Sqrt[-1 + 2*I*Sqrt[2]]]	1 + I*Sqrt[2]	yes	2→1	0.049
A25	Strad[Sqrt[2 + 2*I*Sqrt[3]]]	I + Sqrt[3]	yes	2→1	0.071
A26	Strad[Sqrt[-3 - 2*Sqrt[2]]]	I*(1 + Sqrt[2])	yes	2→1	0.028
A27	Strad[(3 + 2*Sqrt[2])^(1/4)]	Sqrt[1 + Sqrt[2]]	yes	2→2	0.070
A28	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], True]	Sqrt[5 + 2*Sqrt[2]]	yes	3→2	0.090
A29	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]	Sqrt[3 + 2*(1 + Sqrt[2])]	yes	3→2	0.093
A30	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]	yes	3→2	1.165
A31	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]]]	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]	yes	3→2	0.531

Table 17: Corrected version on non-denestable inputs (B) and on the branch counterexamples (C).

ID	Input	Output	Equal	Depth	Time (s)
B01	Strad[Sqrt[2 + Sqrt[2]]]	Sqrt[2 + Sqrt[2]]	yes	2→2	0.041
B02	Strad[Sqrt[1 + Sqrt[2]]]	Sqrt[1 + Sqrt[2]]	yes	2→2	0.023
B03	Strad[(1 + Sqrt[2])^(1/3)]	(1 + Sqrt[2])^(1/3)	yes	2→2	0.069
B04	Strad[Sqrt[2]]	Sqrt[2]	yes	1→1	0.003
B05	Strad[2]	2	yes	0→0	0.001
B06	Strad[Sqrt[1 + I]]	Sqrt[1 + I]	yes	1→1	0.003
B07	Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]	Sqrt[2] + Sqrt[3]	yes	0→1	0.017
B08	Strad[Sqrt[Sqrt[2] + Sqrt[3]]]	Sqrt[Sqrt[2] + Sqrt[3]]	yes	2→2	0.080
B09	Strad[(2 + 2*I*Sqrt[3])^(1/3)]	(2 + (2*I)*Sqrt[3])^(1/3)	yes	2→2	0.269
C01	Strad[-Sqrt[3 + 2*Sqrt[2]]]	-1 - Sqrt[2]	yes	2→1	0.027
C02	Strad[(-1 + 2*I*Sqrt[2])^(3/2)]	-5 + I*Sqrt[2]	yes	2→1	0.043
C03	Strad[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]]]	-((-I + Sqrt[2])*(-I + Sqrt[3]))	yes	2→1	0.138
C04	Strad[(-3 - 2*Sqrt[2])^(3/2)]	(-I)*(7 + 5*Sqrt[2])	yes	2→1	0.054
C05	Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	-Sqrt[2*(7 + 5*Sqrt[2])]	yes	2→2	0.050
C06	Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	-3 - 2*Sqrt[2]	yes	1→1	0.009
C07	Strad[(-1 + 2*I*Sqrt[2])^(5/2)]	1 - (11*I)*Sqrt[2]	yes	2→1	0.045
C08	Strad[(-5 - 2*Sqrt[6])^(3/2)]	(-I)*(11*Sqrt[2] + 9*Sqrt[3])	yes	2→1	0.091

Table 18: Corrected version: direct core calls with forced multipliers (D) and symbolic input (E).

ID	Input	Output	Equal	Depth	Time (s)
D01	DenestCore[-Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {1}]	-1 - Sqrt[2]	yes	2→1	0.029
D02	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {0}]	1 + Sqrt[2]	yes	2→1	0.012
D03	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {}]	1 + Sqrt[2]	yes	2→1	0.006
D04	DenestCore[(-1 + 2*I*Sqrt[2])^(3/2), "Multipliers" -> {1}]	-5 + I*Sqrt[2]	yes	2→1	0.028
D05	DenestCore[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]], "Multipliers" -> {1}]	-((-I + Sqrt[2])*(-I + Sqrt[3]))	yes	2→1	0.089
D06	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, 6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	Sqrt[2] + Sqrt[3]	yes	2→1	0.016

ID	Input	Output	Equal	Depth	Time (s)
D07	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	Sqrt[2] + Sqrt[3]	yes	2→1	0.016
D08	DenestCore[Sqrt[3 + 2*Sqrt[3 + 2* Sqrt[2]]]]	Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]	yes	3→3	0.114
D09	DenestCore[2 + Sqrt[3]]	2 + Sqrt[3]	yes	1→1	0.001
D10	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {2.}]	1 + Sqrt[2]	yes	2→1	0.013
D11	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {Pi}]	1 + Sqrt[2]	yes	2→1	0.013
D12	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, Sqrt[5]}]	Sqrt[2] + Sqrt[3]	yes	2→1	0.013
D13	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {-1}]	1 + Sqrt[2]	yes	2→1	0.012
D14	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {I}]	1 + Sqrt[2]	yes	2→1	0.012
D15	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1}]	Sqrt[2] + Sqrt[3]	yes	2→1	0.019
E01	Strad[Sqrt[z^2]]	Sqrt[z^2]	–	1→1	0.017
E02	Strad[Sqrt[t + t^2]]	Sqrt[t + t^2]	–	1→1	0.002
E03	Strad[Sqrt[-x - y]]	Sqrt[-x - y]	–	1→1	0.002
E04	Strad[Sqrt[a + b]]	Sqrt[a + b]	–	1→1	0.002
E05	Strad[x]	x	–	0→0	0.000
E06	Strad[Sqrt[x]*Sqrt[3 + 2*Sqrt[2]]]	(1 + Sqrt[2])*Sqrt[x]	–	2→1	0.018
E07	Strad[f[1]]	f[1]	–	0→0	0.001
E08	Strad[f[Sqrt[3 + 2*Sqrt[2]]]]	f[1 + Sqrt[2]]	–	2→1	0.014
E09	Strad[RadicalDenest'Private'mark[1]]	1	–	0→0	0.001

Table 19: Corrected version: factorizer and rationalizer probes
(J) and miscellaneous inputs (K).

ID	Input	Output	Equal	Depth	Time (s)
J01	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]	Sqrt[-(-1)^(1/3) - Sqrt[2]]	yes	2→2	0.480
J02	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3)) + 1]]	1 + Sqrt[-(-1)^(1/3) - Sqrt[2]]	yes	2→2	0.437
J06	Strad[(-(Sqrt[2] + (-1)^(1/3)))^(1/3)]	(-(-1)^(1/3) - Sqrt[2])^(1/3)	yes	2→2	1.826
J09	Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/ 3))]]	Sqrt[-(-1)^(1/3) - Sqrt[2]]	yes	2→2	0.002
J10	Factorc[Sqrt[-3 - 2*Sqrt[2]]]	I*Sqrt[3 + 2*Sqrt[2]]	yes	2→2	0.001
J11	Factorc[Sqrt[8 + 4*Sqrt[3]]]	2*Sqrt[2 + Sqrt[3]]	yes	2→2	0.006
J12	Factorc[Sqrt[z^2]]	Sqrt[z^2]	–	1→1	0.001

ID	Input	Output	Equal	Depth	Time (s)
J13	Factorc[Sqrt[-x - y]]	Sqrt[-x - y]	–	1→1	0.002
J14	Factorc[Sqrt[t + t ²]]	Sqrt[t*(1 + t)]	–	1→1	0.001
J15	RationalizeDenominator[x/(1 + Sqrt[2])]	-x + Sqrt[2]*x	–	1→1	0.001
J16	RationalizeDenominator[1/2 + 1/Sqrt[2]]	(1 + Sqrt[2])/2	yes	1→1	0.001
J17	RationalizeDenominator[1/x]	x ⁽⁻¹⁾	–	0→0	0.000
K01	Strad[Sqrt[28 ^(1/3) - 3]]	(-1 - 2 ^(2/3) *7 ^(1/3) + 2 ^(1/3) *7 ^(2/3))/3	yes	2→1	0.121
K02	Strad[(1 + 2 ^(1/3)) ³]	(1 + 2 ^(1/3)) ³	yes	1→1	0.007
K04	Strad[Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True], True]	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]	yes	2→2	1.762
K09	Strad[Sqrt[5 + 2*Sqrt[6]]/Sqrt[3 + 2*Sqrt[2]]]	2 - Sqrt[2] - Sqrt[3] + Sqrt[6]	yes	2→1	0.284
K10	Strad[Exp[I*(Pi/7)]*Sqrt[3 + 2*Sqrt[2]]]	(-1) ^(1/7) *(1 + Sqrt[2])	yes	2→1	0.107
K11	Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]	1 + Sqrt[2] + Pi	yes	2→1	0.035
K12	Strad[Sqrt[3. + 2*Sqrt[2]]]	2.414213562373095	yes	0→0	0.000
K14	Strad[{Sqrt[3 + 2*Sqrt[2]], Sqrt[5 + 2*Sqrt[6]]}]	{1 + Sqrt[2], Sqrt[2] + Sqrt[3]}	–	2→1	0.103
K15	Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]	True	–	0→0	0.002
K17	Strad[Sqrt[(3 + 2*Sqrt[2])/(5 + 2*Sqrt[6])]]	-2 - Sqrt[2] + Sqrt[3] + Sqrt[6]	yes	2→1	0.206
K19	Strad[Sqrt[Sqrt[2] + 1] ³]	(1 + Sqrt[2]) ^(3/2)	yes	2→2	0.065
K20	Strad[Sqrt[2 + Sqrt[3]]*Sqrt[3 + 2*Sqrt[2]]]	((2 + Sqrt[2])*(1 + Sqrt[3]))/2	yes	2→1	0.148
K21	Strad[Sqrt[3 + 2*Sqrt[2]], "Verbose" -> False, "MaxTrials" -> 1]	1 + Sqrt[2]	yes	2→1	0.035
K22	Strad[Sqrt[5 + 2*Sqrt[6]], "Factor" -> False]	Sqrt[2] + Sqrt[3]	yes	2→1	0.041

Table 20: Corrected version: higher-order roots (L) and roots of negative bases (Q).

ID	Input	Output	Equal	Depth	Time (s)
L01	Strad[(239 + 169*Sqrt[2]) ^(1/7)]	1 + Sqrt[2]	yes	2→1	0.067
L02	Strad[(2 + Sqrt[2]) ^(1/5)]	(2 + Sqrt[2]) ^(1/5)	yes	2→2	0.430
L03	Strad[(1 + Sqrt[2]) ^(1/7)]	(1 + Sqrt[2]) ^(1/7)	yes	2→2	0.773
L04	Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]	Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7]	yes	2→1	2.916

ID	Input	Output	Equal	Depth	Time (s)
L05	Strad[Expand[(1 + Sqrt[2] + Sqrt[3]) ³] ^(1/3)]	1 + Sqrt[2] + Sqrt[3]	yes	2→1	0.118
L06	Strad[Expand[(Sqrt[2] + Sqrt[3]) ⁴] ^(1/4)]	Sqrt[2] + Sqrt[3]	yes	2→1	0.057
L07	Strad[Sqrt[1000003 + 2*Sqrt[999983]]]	Sqrt[1000003 + 2*Sqrt[999983]]	yes	2→2	0.212
L08	Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]	Sqrt[2 + Sqrt[3] + Sqrt[5]]	yes	2→2	0.376
L09	Strad[(5 + 2*Sqrt[6]) ^(1/6)]	(5 + 2*Sqrt[6]) ^(1/6)	yes	2→2	1.554
L10	Strad[(11*Sqrt[2] + 9*Sqrt[3]) ^(1/3)]	Sqrt[2] + Sqrt[3]	yes	2→1	0.054
L11	Strad[Expand[(2 ^(1/3) + 3 ^(1/3)) ³] ^(1/3)]	2 ^(1/3) + 3 ^(1/3)	yes	2→1	0.266
L12	Strad[Sqrt[Expand[(1 + 2 ^(1/3) + 3 ^(1/3)) ²]]]	1 + 2 ^(1/3) + 3 ^(1/3)	yes	2→1	0.058
L13	Strad[Expand[(1 + 2 ^(1/5)) ⁵] ^(1/5)]	1 + 2 ^(1/5)	yes	2→1	0.055
L14	Strad[Sqrt[5 + 2*Sqrt[6]] ^(1/5)]	(5 + 2*Sqrt[6]) ^(1/10)	yes	2→2	6.605
Q01	Strad[Expand[(-1 - Sqrt[2]) ³] ^(1/3)]	((1 + Sqrt[2])*(1 + I*Sqrt[3]))/2	yes	2→1	0.401
Q02	Strad[Expand[(1 - Sqrt[2]) ³] ^(1/3)]	((-1 + Sqrt[2])*(1 + I*Sqrt[3]))/2	yes	2→1	0.366
Q07	Strad[Expand[(2 - 3*Sqrt[2]) ³] ^(1/3)]	((-I)*(-3 + Sqrt[2])*(-I + Sqrt[3]))/Sqrt[2]	yes	2→1	0.475
Q08	Strad[(-7 - 5*Sqrt[2]) ^(1/3)]	((1 + Sqrt[2])*(1 + I*Sqrt[3]))/2	yes	2→1	0.212

6 Unified catalogue of defects and shortcomings

Each entry gives the severity, the source lines, the reviews that report it, the experimental evidence, and the repair adopted in `StradFixed.wl` (Section 7). Severity: **critical** = wrong value can be returned; **high** = crash, garbage output, or uncontrolled resource use; **medium** = wrong or misleading behaviour without a wrong value; **low** = maintenance.

6.1 Wrong values

U1 The candidate filter certifies against $(\beta^q)^{1/q}$ instead of β

Severity: critical. *Source:* lines 94, 125–127. *Reviews:* R1 F01, R2 B1, R3 F1.

Evidence. D01, D04, D05 (engine), C02, C03 (wrapper); fuzz F5 39/40 and F6 17/40 wrong. The wrapper’s grouping rules and its acceptance of exponents p/q with $p > 1$ deliver exactly the values outside the principal sector of Proposition 3.3. The reviews’ hedge that a leading minus sign is kept outside the marker is confirmed (C01 is right). *Repair.* `tryMultiplier` receives `problem` and selects with `CertifiedEqualQ[##, problem]`; the orbit of Proposition 3.2 is unchanged.

U2 The custom factorizer applies power identities without branch conditions

Severity: critical. *Source:* lines 527–531, 571–583. *Reviews:* R1 F02/F03, R2 B2, R3 F2.

Evidence. Symbolic: E01 $\sqrt{z^2} \rightarrow z$, E03 $\sqrt{-x-y} \rightarrow i\sqrt{x+y}$, H18 $\sqrt{x^4} \rightarrow x^2$, H19 $(x^2)^{1/3} \rightarrow x^{2/3}$. Numeric, end to end: J01/J02 $\sqrt{-(\sqrt{2} + (-1)^{1/3})} \rightarrow i\sqrt{(-1)^{1/3} + \sqrt{2}}$ (the negative of the input), J06 the cube-root analogue. R1’s symbolic probe `Sqrt[t + t^2]` does *not* fail (H02, and the passing test in its own suite). *Repair.* Proposition 3.5: the exponent merge is applied only when the outer exponent is an integer or the inner exponent is real in $(-1, 1]$; common factors are pulled out of a sum under a fractional power only when the power is an integer, the extracted factor is a positive real, or the remaining sum is a positive real. Factoring is applied only to exact numeric subexpressions and its output is certified against its input.

6.2 Unsafe acceptance and validation

U3 PossibleZeroQ with default method is the only certificate

Severity: high. *Source:* lines 125–127. *Reviews:* R1 F04, R2 §8.3, R3 F6.

Evidence. In 20 of 31 classical rows and 272 of 340 random inputs the kernel reports `PossibleZeroQ::ztest1`: it could not decide and assumed zero. Because the candidates are exact algebraic numbers and the correct one is present in the orbit, this “assumption” happened to be right in every observed case, and S04–S06 show that the default method does reject close non-zero differences. The defect is a missing guarantee, not an observed false positive. *Repair.* `CertifiedEqualQ`: 30-digit numeric rejection, then `RootReduce[a - b] === 0`, then `PossibleZeroQ[... , Method -> "ExactAlgebraics"]`, each under a time limit; undecided means rejected. No message is emitted.

U4 Zero, inexact or non-algebraic multipliers and empty candidate lists are not handled

Severity: high. *Source:* lines 111–134, 214–215. *Reviews:* R1 F05, R2 B5/B10, R3 F5.

Evidence. D02 returns `{}[[1]]` after printing “success”; D10 returns `Outer[][[1]]`; both leave `Power::infty` and `Part::partw` messages. *Repair.* Multipliers must be exact, algebraic and nonzero; polynomial results are type-checked; an empty certified list is an ordinary “no candidate” outcome.

U15 The input domain is not enforced

Severity: high. *Source:* lines 5, 79, 454. *Reviews:* R1 F10, R2 B10, R3 F9.

Evidence. K14: a list of two radicals becomes $(3 + 2\sqrt{2})^{(\sqrt{2}+\sqrt{3})/2}$ because the nesting helpers and the factorizer encode structure as lists. K12: an inexact input is returned as a machine number. B07: a `Root` object becomes a nested radical. E08: a user function around a radical triggers seven messages. *Repair.* Lists are mapped over; inexact input is returned unchanged; `Root` input is converted with `ToRadicals` first and then denested; only exact numeric algebraic subexpressions are factored and marked.

6.3 Search control

U5 The prime comparator compares a pair with itself

Severity: high. *Source:* lines 181. *Reviews:* R1 F06, R2 B8, R3 F3.

Evidence. S01: `Sort` with the predicate `##1[[1]] <= ##[[2]]` & returns the list *reversed* (the predicate is `False` for every pair of plain primes and the merge sort then reverses). Consequences (N01): the ratio-pruning loop of lines 184–189 never runs, and the prefix of lines 190–194 keeps the largest primes. *Repair.* `SortBy[... , First]`.

U6 The prefix length uses the pre-deletion length

Severity: high (latent). *Source:* lines 190–194. *Reviews:* R3 F4.

Evidence. S02 and N02: with the comparator repaired, the fixture $\{2, 3, 5, 7, 11, 2003\}$ triggers `Part::take`. In the original program the zeroing loop never runs (U5), so the defect is unreachable; it becomes live as soon as U5 is fixed in isolation. *Repair.* Delete first, then take `Min[count, Length]`.

U7 The multiplier “cap” is not a cap; `Ceiling[Log]` misfires

Severity: high. *Source:* lines 162–199. *Reviews:* R1 F07, R2 B6, R3 F7.

Evidence. Proposition 3.7; N05/N06 materialize 1024 and 100 000 divisors; the largest batch observed in practice was 63 (L04). For $q = 10$ the exact quotient $\text{Log}[1000]/\text{Log}[10]$ is not decided by `Ceiling` (R01–R04, L14), so `Max[5, ...]` silently falls back to 5. *Repair.* $t = \max\{t : q^t - 1 \leq \text{cap}\}$ by integer arithmetic; the post-first-pass product batch is truncated to the cap; a trial budget and a time budget bound the whole search.

U12 The equal-degree gate skips a linear GCD

Severity: medium. *Source:* lines 115. *Reviews:* R2 H1.

Evidence. D06 versus D07. *Repair.* The GCD is attempted when $d_m \leq d_{\text{best}}$.

U13 `List @@ Expand[...]` assumes a sum

Severity: low. *Source:* lines 224–229. *Reviews:* R1 §5.1, R2 H4, R3 §6.1.

Evidence. Not reachable with exact numeric radicands: `Expand` of a power of a nested radical is a `Plus`, and monomial radicands are split by the kernel’s automatic evaluation before they reach the engine (`Sqrt[3 Sqrt[2]]` evaluates to $2^{(1/4)} \text{Sqrt}[3]$). *Repair.* An explicit `summands` helper.

U14 Composite cofactors of the discriminant are dropped

Severity: low. *Source:* lines 173–181. *Reviews:* R1 §4.4, R2 H2, R3 §6.3.

Evidence. Never triggered in the experiments (no “composite factor” diagnostic was printed). *Repair.* Kept, but logged.

U16 No global budget and no visited set

Severity: medium. *Source:* lines 251–432. *Reviews:* R1 F12, R2 B7, R3 §9.3.

Evidence. Duplicate multipliers across batches are visible in the traces (D08 tries 2 and $2\sqrt{3+2\sqrt{2}}$, then 17, 34, $17\sqrt{\cdot}$, $34\sqrt{\cdot}$); no runaway was observed on the tested inputs. *Repair.* "MaxTrials", "TimeBudget", and a visited association keyed by the multiplier.

U22 Single-level mode cannot exploit inner denestings

Severity: medium. *Source:* lines 7–10, 58–61. *Reviews:* all, in passing.

Evidence. A29 (single level) leaves $\sqrt{3+2\sqrt{3+2\sqrt{2}}}$ alone after 12 trials; A28 (all levels) finds $\sqrt{5+2\sqrt{2}}$. *Repair.* The default remains single level for compatibility, but a partial solution of depth ≥ 2 is passed through one further all-level pass.

U23 Candidates contain denestable inner surds

Severity: medium. *Source:* lines 121–134. *Reviews:* –.

Evidence. Q01–Q08: the roots of the quadratic GCD contain $\sqrt{9-6\sqrt{2}} = \sqrt{6}-\sqrt{3}$ and **Simplify** does not remove it, so 17 of 30 negative-base cube roots keep their depth. *Repair.* Candidate polishing applies Proposition 3.6 to inner square roots, then **Simplify**; all 30 are denested.

U24 “Forced” multipliers are only initial multipliers

Severity: low. *Source:* lines 214–215, 338–378. *Reviews:* all.

Evidence. D12: with $\{1, \sqrt{5}\}$ the degree-8 trial counts as progress and a discriminant batch finds 2. *Repair.* Documented as the "Multipliers" option with the same semantics.

U25 The search is heuristic and incomplete

Severity: medium. *Source:* lines 159–249. *Reviews:* R1 F11, R2 H3, R3 F11.

Evidence. No false “denesting exists” claims were observed; A13 and the F3 controls are correctly unchanged. No completeness certificate is possible for this design. *Repair.* None; see Section 8.

6.4 Output quality**U9 “Success” is not measured against an objective**

Severity: medium. *Source:* lines 120–134, 309–324, 435–438. *Reviews:* R1 F09, R2 H5, R3 F8.

Evidence. D07 returns $(1 + \sqrt{2} + \sqrt{3})/\sqrt{2(3 - \sqrt{2} + \sqrt{3} - \sqrt{6})}$ for $\sqrt{5+2\sqrt{6}}$; A30/A31 and L04 return correct but huge quotients; B01 rewrites $\sqrt{2+\sqrt{2}}$ as $2^{1/4}\sqrt{1+\sqrt{2}}$. *Repair.* The cost $\text{cost}(E) = (\#Root, rdepth, \#radical \text{ nodes}, \#atoms, LeafCount)$ is compared lexicographically; a result is returned only if it is strictly cheaper than the input. Candidates are polished by **Simplify**,

rationalization of the denominator (which turns the L04 quotient into $\sqrt{2} + \sqrt{3} + \sqrt{5} + \sqrt{7}$), and inner-surd denesting.

U20 The final `Simplify` re-nests `Root` objects and rewrites unchanged input

Severity: medium. *Source:* lines 59–61, 73. *Reviews:* R3 in passing.

Evidence. B07, B01, and the 8 “equal, rewritten” F3 outputs. *Repair.* `Simplify` is one of several candidate variants subject to the cost gate, and `Root` input is converted *before* denesting.

U27 Quotient candidates are not rationalized

Severity: low. *Source:* lines 121–134. *Reviews:* –.

Evidence. L04; the program’s own `RationalizeDenominator` followed by `Simplify` produces the intended answer (S01 in Table 13). *Repair.* Part of candidate polishing.

6.5 Interfaces and session hygiene

U8 Loading the file unprotects `Print`

Severity: medium. *Source:* lines 2. *Reviews:* R1 F08, R2 B9, R3 F10.

Evidence. `Attributes[Print]` is `{Protected}` before and `{}` after `Get`. *Repair.* Verbose logging through an option.

U10 The marker `f` collides with user symbols

Severity: high. *Source:* lines 6, 12, 55. *Reviews:* R2 B3.

Evidence. M01/E07 (`f[1]` becomes 1), E08. *Repair.* A private-context marker symbol. E09 shows that even the private marker is handled gracefully.

U11 `RationalizeDenominator` substitutes `x -> 0` into the numerator

Severity: high. *Source:* lines 445–450. *Reviews:* R2 B4.

Evidence. R05 returns 0 for `x/(1 + Sqrt[2])`. *Repair.* Module-local variable; non-algebraic denominators are returned unchanged (J17).

U19 `Strad` suppresses `Print` but not messages

Severity: medium. *Source:* lines 3. *Reviews:* –.

Evidence. Ordinary successful calls show `N::meprec` and `PossibleZeroQ::ztest1`; invalid input shows `Power::infy`, `Part::partw`, and others. *Repair.* No message-producing operation remains on the normal path; failures are caught and turned into “no candidate”.

U17 Fragile positional state; `Throw[, "solution"]`

Severity: low. *Source:* lines 253–267, 292, 320. *Reviews:* R1 F13, R2 M1, R3 §9.5.

Evidence. Loading emits `Syntax::com` for [lines 320](#). *Repair.* `Throw[Null, "solution"]`; otherwise the record layout is kept for fidelity, with type checks where records are consumed.

U18 Dead settings and unused variables

Severity: low. *Source:* [lines 95–103, 163, 400](#). *Reviews:* R1 F14, R2 M1, R3 F12.

`extrapower, maxprimes, best` (in the extra block), `expected, done`. *Repair.* Removed.

U21 combine does not clean a trailing unit entry

Severity: low. *Source:* [lines 558–568](#). *Reviews:* R3 §9.5.

Repair. Loop bound `j <= Length`.

U26 All-level mode marks every radical, including $\sqrt{29}$

Severity: low. *Source:* [lines 9, 11](#). *Reviews:* –.

Evidence. The A30 trace shows `f[Sqrt[29]]` as a “problem”. Harmless: the engine returns it at its gate.

6.6 What is *not* a defect

The sign-flip loop with its accumulator ([lines 30–37](#)) preserves the product; one-argument `MinimalPolynomial` returning a pure function is valid; non-strict comparators are permitted by `Sort`; `Extension -> Automatic` handles nested radicands (D08); R3’s level-0 `MemberQ` remark does not apply to exact numeric input; and the rationalizer’s formula is correct including the non-monic case (R01–R10). The engine’s search order, though heuristic, found every classical denesting we tried, usually with the first or second discriminant batch.

7 The corrected version

`src/corrected/StradFixed.wl` (Appendix B, about 480 lines) is a package in the context `RadicalDenest’`. It deliberately keeps the original architecture (wrapper, markers, grouping, multiplier search, history and stack, discriminant batches, the post-first-pass product batch) so that the search behaviour stays recognizable, and changes only what the catalogue requires. The design rule is the one all three reviews recommend: *heuristics propose, exact algebra disposes*.

7.1 Interface

```
Strad[expr] (* single level, as before *)
Strad[expr, True] (* all levels, as before *)
Strad[expr, "AllLevels" -> True, "Verbose" -> True]
DenestCore[problem, "Multipliers" -> {2}] (* the engine, formerly denest11 *)
```

Options: `"AllLevels"` (False), `"Verbose"` (False), `"Multipliers"` (Automatic or an initial list), `"MaxTrials"` (400), `"TimeBudget"` (120 s), `"MultiplierCap"` (1000), `"CertifyTime"` (20 s per

exact decision), "Solver" (Automatic or a function replacing the engine), "Factor" (True). Lists are mapped over. Public helpers: RationalizeDenominator, Factorrc, RadicalDepth, RadicalCost, ExactAlgebraicQ, CertifiedEqualQ.

7.2 Change list

1. **Certification** (U1, U3). `CertifiedEqualQ[a, b]` returns `True` only if $a - b$ is proved zero: exact inputs, a 30-digit numeric rejection, `RootReduce`, then `PossibleZeroQ` with `Method -> "ExactAlgebraics"`; every step is time limited and "undecided" is "not equal". The engine's filter is `Select[possibilities, CertifiedEqualQ[##, problem] &]`. The wrapper certifies its whole result against the whole input whenever the input is an exact number.
2. **Branch-safe factorizer** (U2). `fullfactor` merges $\{a^b, c\}$ into $\{a, bc\}$ only if `powerMergeSafeQ[b, c]` (Proposition 3.5(1)); `combine` extracts common factors from a sum under exponent p only if $p \in \mathbb{Z}$, the extracted product is a positive real, or the remaining sum is a positive real (Proposition 3.5(2)); `Factorrc` certifies its own output; the wrapper factors only maximal exact numeric non-atomic subexpressions. The useful rewrites survive: J10 $\sqrt{-3 - 2\sqrt{2}} \rightarrow i\sqrt{3 + 2\sqrt{2}}$ and J11 $\sqrt{8 + 4\sqrt{3}} \rightarrow 2\sqrt{2 + \sqrt{3}}$.
3. **Validation** (U4, U15). Exact algebraic nonzero multipliers; integer root order ≥ 2 ; polynomial-type checks after `MinimalPolynomial`, `PolynomialGCD` and `Solve`; empty candidate sets are ordinary; inexact input is returned; `Root` input goes through `ToRadicals` first; lists are mapped.
4. **Search control** (U5, U6, U7, U12, U16). `SortBy[... , First]`; delete-then-take; the cap $q^t - 1 \leq \text{cap}$ in integer arithmetic; the GCD attempted for $d_m \leq d_{\text{best}}$; a visited set; "MaxTrials" and "TimeBudget" checked before every trial; the product batch after the first pass truncated to the cap; `Throw[Null, ...]`.
5. **Objective** (U9, U20, U27). `RadicalCost` and `cheaperQ`; the engine keeps the cheapest certified candidate and only replaces a solution by a cheaper one; the wrapper returns the cheapest of $\{\text{input, result, Simplify[result], inner-surd variant}\}$ that is certified equal to the input, and never anything more expensive than the input.
6. **Candidate polishing** (U23, U27). Each certified candidate is offered in the variants $\{\text{as is, Simplify, rationalized denominator, inner surds denested by Proposition 3.6, both}\}$, and the cheapest is kept.
7. **Quadratic-surd fast path** (R3's proposal). A problem of the form $\sqrt{a + b\sqrt{c}}$ with rational a, b, c is tried with Proposition 3.6 before any minimal polynomial is computed; the result is certified anyway.
8. **One recursive pass** (U22). A partial solution of depth ≥ 2 is sent once through an all-level pass (recursion depth at most 2), so $(49 + 20\sqrt{6})^{1/4} \rightarrow \sqrt{5 + 2\sqrt{6}} \rightarrow \sqrt{2} + \sqrt{3}$.
9. **Hygiene** (U8, U10, U11, U17, U18, U19, U21). Package context; private marker; `Module` variables; no `Unprotect`; `log[...]` instead of `Print`; dead code removed; the trailing unit entry cleaned.

7.3 What was deliberately kept

The initial-multiplier heuristic, the degree-based “progress” logic, the history ordering, the stack discipline (split-and-insert for partial batches, deferred insertion for full-set batches), the discriminant batches with prime-gap trimming, the product batch after the first pass, and the early stopping rule are all retained. They are heuristics that the experiments show to be effective on the classical corpus, and replacing them was not the goal of a *corrected* version. Their remaining weaknesses are listed in Section 8.

7.4 Guarantees and their limits

For an exact algebraic input E the corrected version guarantees: (i) the result is certified equal to E by `RootReduce` or exact `PossibleZeroQ`; (ii) the result is strictly cheaper than E under `RadicalCost`, or is E itself; (iii) no kernel message is produced on the normal path; (iv) the search stops after “MaxTrials” trials or “TimeBudget” seconds. For an input with symbolic parts, (i) holds for every replaced numeric subexpression individually and the symbolic host is left untouched. There is no completeness guarantee: an unchanged result means that the bounded search found nothing cheaper, not that nothing exists. Certification can time out on very large expressions, in which case candidates are rejected and the input is returned; this is the conservative direction.

7.5 Cost

The corrected version is two to three times slower than the original on the small inputs of the battery (Table 16 versus Table 4): almost all of the difference is `RootReduce` inside `CertifiedEqualQ`, called for the Factorc check, for each candidate that passes the numeric prefilter, and once more for the whole result. The slowest probe, $\sqrt{5 + 2\sqrt{6}}^{1/5}$, takes 7 s (20 s in the original).

8 Future improvements

The corrected version makes the program *safe*; it does not make it *complete* or *fast*. The following improvements are listed in the order in which we would undertake them.

8.1 Algorithmic

1. **Replace degree heuristics by field computations.** The engine guesses multipliers and asks a degree oracle whether it guessed well. For a problem β with $\rho = \beta^q$, the question “does $X^q - \rho$ have a linear factor over $K = \mathbb{Q}(\rho)$?” is answered directly by `Factor[x^q - rho, Extension -> Automatic]`; a linear factor is a denesting into K with no multiplier at all, and the trial $m = 1$ is then redundant. More generally, for β of degree n over $\mathbb{Q}$ the candidate fields for a depth reduction are the subfields of the Galois closure; Landau’s algorithm for nested square roots and Blömer’s algorithms for real radicals and for roots of unity reduce the problem to computing a specific extension (e.g. adjoining $\sqrt[q]{d}$ for a *single* well-chosen d derived from the discriminant or from the norm of ρ) and testing membership. The discriminant-prime batches of `newmultipliers` are a heuristic approximation of this; the exact version would try $m = \pm d^e$ for the q -th-power-free divisors d of the norm $N_{K/\mathbb{Q}}(\rho)$ and its conjugates, which

is a finite and provably sufficient list for Kummer-type denestings ($\sqrt[q]{\rho} \in K(\sqrt[q]{m})$ for some $m \in K$).

2. **A complete quadratic-surd layer.** Proposition 3.6 covers the case $a^2 - b^2c = \delta^2$. The other half of Landau's theorem, $c(a^2 - b^2c) = \delta^2$, yields denestings with a fourth root ($\sqrt{a + b\sqrt{c}} = c^{1/4}\sqrt{\dots}$ forms); and nested square roots over a real quadratic field $\mathbb{Q}(\sqrt{d})$ admit an exact decision procedure (Borodin–Fagin–Hopcroft–Tompkins) that would find A30 in one step instead of a two-level multiplier search. Both are cheap to add in front of the engine.
3. **Denest sums and products jointly.** The wrapper treats $\sqrt{2 + \sqrt{3}} + \sqrt{2 - \sqrt{3}}$ (A23) as two problems and relies on `Simplify` to combine the answers. A joint treatment ($\beta = \beta_1 + \beta_2$ with $\beta^2 \in \mathbb{Q}(\sqrt{3})$) is the Borodin–Fagin–Hopcroft–Tompkins setting and would also cover $\sqrt{a + \sqrt{b}} + \sqrt{a - \sqrt{b}}$ patterns that the current design cannot see.
4. **Root objects as first-class citizens.** The corrected version converts `Root` input to radicals and forbids `Root` in the output by its cost. A “canonical” mode could instead accept `Root` output (`RootReduce` is the canonical form) and use `ToRadicals` only when it yields a radical-only expression of lower depth.
5. **Roots of unity and complex bases.** Cube roots of negative reals denest to $e^{i\pi/3}(\dots)$; the corrected version finds these but prints them as $(1 + i\sqrt{3})/2 \cdot (\dots)$. A normal form that writes $(-1)^{p/q}$ factors explicitly and a fast path for $\rho = -|\rho|$ would give shorter answers in one step.

8.2 Search engineering

1. **Lazy, prioritized generation.** Batches are still materialized as lists (bounded by the cap). A priority queue of *generators* (exponent vectors over the selected primes, ordered by a cost such as $\sum e_i \log p_i$) would remove the cap altogether and make the trial order independent of `Divisors`'s ordering.
2. **Caching.** `MinimalPolynomial` of $(m\rho)^{1/q}$ is the dominant cost per trial and is recomputed for algebraically equal multipliers written differently. Key the visited set by `RootReduce[m]` (with a small time limit) instead of by syntax.
3. **Cheaper certification.** `RootReduce` is called on differences that are exactly zero, which is its worst case. A modular or interval prefilter with a proven bound (e.g. compare at 200 digits and only then call `RootReduce`), or certification by `MinimalPolynomial[candidate] == MinimalPolynomial[problem]` plus a numeric separation argument between conjugates, would cut the factor of two to three observed in Section 5.6.
4. **Structured results.** The engine should return an association (status, best certified value, cost, trials used, budget exhausted?, multipliers tried) so that callers can distinguish “nothing found” from “budget exhausted” and “certification timed out”. The current corrected version returns only the expression for compatibility.
5. **Records instead of positional lists.** The history and stack layout of the original is preserved; replacing it by associations with explicit keys (`"Multiplier"`, `"Degree"`, `"Batch"`, `"Deferred"`) would let the invariants be checked and the scheduling be unit-tested with a mock trial function.
6. **Termination.** The outer loop compares its index with a growing stack. The budget makes

termination certain, but a monotone measure (each inserted batch strictly decreases the best degree or is a product batch inserted at most once) would make it provable without a budget.

8.3 Testing and quality

1. **Property-based tests from constructed answers.** Every family of Section 5.3 is of the form “build β , hand over β^q raised to $1/q$ ”; the same generator can produce nested answers ($\beta = \sqrt{5} + \sqrt{11 - 2\sqrt{29}}$), roots of unity, and products, and the oracle is exact equality plus a depth target.
2. **Regression suite.** The harness of Appendix C and the reviews’ three `.wlt` files should be merged into one `TestReport` suite run on both versions; all expected values used here were verified with `RootReduce` in the kernel.
3. **Timing corpus.** The battery is small-input only. A corpus of degree-16 to degree-64 problems (four to six independent square roots, cube roots of sums of three cube roots) is needed before any claim about performance; the original’s 20s tenth root suggests where the cliff is.
4. **Output normal form.** Choose and document one: the corrected version prefers fewer radical nodes and then fewer atoms, which yields $\sqrt{3/2} + 1/\sqrt{2}$ for A03 where the original’s `Simplify` gave $(1 + \sqrt{3})/\sqrt{2}$. Both are depth 1; a documented tie-break (e.g. rationalized denominators, integer radicands) would remove the surprise.

9 Conclusion

The three reviews were right about the essentials and, being static, could not see the rest. Executed in Wolfram 15.0.1, `Strad.wl` denests every classical identity in our corpus quickly and correctly, and its two mathematical ideas, the multiplier-driven polynomial GCD over an algebraic extension and the minimal-polynomial rationalizer, are sound. It also returns wrong values on a large, natural class of inputs: 56 of 340 random inputs, concentrated entirely in complex products and non-principal powers, plus a purely numeric case caused by its own factorizer. Its acceptance test rests on `PossibleZeroQ` assuming zero when it cannot decide; its prime comparator reverses the prime list and thereby disables its own pruning while hiding a second bug; it mishandles lists, inexact numbers and `Root` objects; and several of its “successes” are larger than the input.

The unified catalogue contains 27 items. Sixteen were reported by at least one review and are confirmed here; five were reported and are refined (one is unreachable, one is moot, one probe was mis-chosen, two are practically inert); six are new. The corrected package removes every wrong value observed and every message, adds budgets and an explicit improvement criterion, and finds several denestings the original misses, at a cost of two to three in running time. Everything reported is reproducible from the harness, logs and scripts shipped alongside this document.

Unified	Short description	R1	R2	R3	Status
U1	Candidates certified against $(\beta^q)^{1/q}$, not β	F01	B1	F1	confirmed, quantified
U2	Branch-unsafe power rules in the factorizer	F02, F03	B2	F2	confirmed; new numeric case
U3	PossibleZeroQ as certificate	F04	o	F6	confirmed: “assuming zero” in practice
U4	Zero/invalid multipliers, <code>{ }[[1]]</code>	F05	B5, B10	F5	confirmed
U5	Comparator <code>##1[[1]]</code> <code><= ##[[2]]</code>	F06	B8	F3	confirmed; reverses the list
U6	Stale length in the prime slice	–	–	F4	confirmed snippet; <i>unreachable</i> in original
U7	Cap is not a cap; <code>Ceiling[Log]</code>	F07	B6	F7	confirmed; new <code>Ceiling::meprec</code>
U8	<code>Unprotect[Print]</code> at load	F08	B9	F10	confirmed
U9	“Success” is not an improvement	F09	H5	F8	confirmed; several ugly outputs
U10	Marker <code>f</code> collides with user data	–	B3	–	confirmed
U11	Rationalizer’s <code>x -> 0</code> erases a numerator	–	B4	–	confirmed
U12	Equal-degree gate skips a linear GCD	–	H1	o	confirmed
U13	<code>List @@ Expand</code> head assumption	o	H4	o	not reachable with exact input
U14	Composite discriminant cofactors dropped	o	H2	o	theoretical; not observed
U15	Input domain unenforced (lists, inexact, <code>Root</code>)	F10	B10	F9	confirmed: list input is nonsense
U16	No budget, no visited set	F12	B7	o	confirmed duplicates; no blow-up seen
U17	Fragile positional state; <code>Throw[,]</code>	F13	M1	o	new: load-time <code>Syntax::com</code>
U18	Dead settings	F14	M1	F12	confirmed by inspection
U19	Messages leak through <code>Strad</code>	–	–	–	new
U20	Final <code>Simplify</code> re-nests <code>Root</code> , rewrites non-denested input	–	–	o	new
U21	<code>combine</code> skips the last unit entry	–	–	F11	confirmed by inspection
U22	Single-level mode cannot use inner denestings	o	o	o	confirmed (A29 vs A28)
U23	Candidates contain denestable inner surds	–	–	–	new: 17/30 cube roots not denested
U24	“Forced” multipliers are only initial	o	o	o	confirmed (D12)
U25	Search incompleteness in general	F11	H3	F11	theoretical; A13 correctly unchanged
U26	All-level mode marks trivial radicals	–	–	–	new, harmless
U27	Poor final form (quotients, unrationalized)	o	o	o	new: 4-term and BFHT outputs

Family	n	Original				Corrected		
		denested	equal, rewritten	unchanged	wrong	denested	unchanged	wrong
F1 $\sqrt{(p + q\sqrt{c})^2}$	40	40	0	0	0	40	0	0
F2 $((p + q\sqrt{c})^3)^{1/3}$	30	13	17	0	0	30	0	0
F3 $\sqrt{a + b\sqrt{c}}$, random	40	6	8	26	0	6	34	0
F4 $\sqrt{(p + qi\sqrt{c})^2}$	40	40	0	0	0	40	0	0
F5 $((p + qi\sqrt{c})^2)^{3/2}$	40	1	0	0	39	40	0	0
F6 $\sqrt{u^2}\sqrt{v^2}$, complex	40	23	0	0	17	40	0	0
F7 $((p + q\sqrt{c})^4)^{1/4}$	25	17	8	0	0	25	0	0
F8 $\sqrt{(p + q\sqrt{c} + r\sqrt{d})^2}$	25	25	0	0	0	25	0	0
F9 $((p + q\sqrt{c})^2)^{3/2}$, real	30	30	0	0	0	30	0	0
F10 $\sqrt{-u^2}\sqrt{-v^2}$, real	30	30	0	0	0	30	0	0
Total	340	225	33	26	56	306	34	0

Table 14: Randomized families. The original program emitted `PossibleZeroQ::ztest1` on 272 of the 340 inputs; the corrected version emitted no message. Wall-clock totals: original 27 s, corrected 39 s; the slowest single input took 0.4 s (original) and 1.7 s (corrected).

Suite	Failing tests (all predicted by the review)	Passing
R1 (14)	Print protection; comparator; negative branch; grouped complex branch; wrapper complex product; <code>Factorc[Sqrt[t^2]]</code> ; zero multiplier	7: rationalizer (2), direct and multiplier success, <code>Sqrt[t + t^2]</code> (<i>not</i> predicted to pass), empty seed list, all-levels
R2 (15)	comparator; negative branch; complex 3/2 power; merged product; <code>Factorc[Sqrt[z^2]]</code> ; caller's <code>f</code> ; numerator <code>x</code>	4 on the original (Print protection ¹ , worked example, rationalizer, all-level control); the 4 tests of its guarded component pass when the component is loaded (Section 5.5)
R3 (20)	negative core target; complex 3/2; merged product (core and wrapper); <code>Factorc[Sqrt[-x-y]]</code> ; zero multiplier; comparator; stale slice; cap; Print protection	10: classical success, rationalizer (2), non-nested input, wrapper modes with <code>Identity</code> , and all 5 tests of its proposed kernel

Table 15: The reviews' regression suites executed against the original program.

A The original source, with line numbers

The file `src/original/Strad.wl` is reproduced verbatim (585 physical lines). All “lines n – m ” references in the text refer to these numbers.

```

1 ClearAll[Strad];
2 Unprotect[Print];
3 Strad[eaf_] := Block[{Print = (Null &)}, DenestRadicals3[eaf]];
4 ClearAll[DenestRadicals3];
5 DenestRadicals3[expr_, alllevels_ : False, func_ : denest11] :=
6 Block[{f, mod, level, problems, i1, i2},
7   If[alllevels != True,
8     level = ReplaceAll,
9     level = ReplaceRepeated
10  ];
11  mod = level[Factorc[expr],
12    radicand_?AlgebraicQ^pow_Rational := Block[{factored, SumQ, i, j},
13      SumQ[expression_] :=
14        MemberQ[
15          expression, _Plus | Complex[Except[0], _], {0, Infinity}];
16      factored = FactorTermsList[radicand];
17      factored[[2]] =
18        Flatten[{Replace[Factorc[factored[[2]]],
19          prod_Times := List @@ prod]}];
20      factored[[1]] =
21        DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[
22          GCD[Re[factored[[1]]], Im[factored[[1]]]], 1];
23      factored[[1]] = #[[1]]^#[[2]] & /@
24        Flatten[FactorInteger /@ factored[[1]], 1];
25      factored = Flatten[factored];
26      factored =
27        factored //.
28        list : {a___, expr1_, b___, expr2_, c___} := {a, b, c,
29          ComplexExpand[expr1*expr2]} /; And[SumQ[expr1], SumQ[expr2]];
30      j = 1;
31      For[{i = 1}, i <= Length[factored], i++,
32        If[N[Re[factored[[i]]], 100] < 0,
33          factored[[i]] = -factored[[i]];
34          j *= -1
35        ];
36      ];
37      factored = DeleteCases[Prepend[factored, j], 1];
38      f[Simplify[
39        Times @@ (Simplify[RationalizeDenominator[Together[#]]] & /@
40          factored)], pow]
41    ]
42  ];
43  mod = mod //.
44    prod : Times[a___, expr1 : f[rad1_, pow1_], b___,
45      expr2 : f[rad2_, pow2_], c___] :=
46      Times[a, b, c, f[(rad1^pow1)*(rad2^pow2)]] /;
47      And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
48  mod = level[mod, f[rad_, pow_] := f[rad^pow]];
49  mod = mod //.
50    prod : Times[a___, expr1 : f[rad1_], b___, expr2 : f[rad2_],
51      c___] :=
52      Times[a, b, c, f[rad1*rad2]] /;
53      And[And[Re[#] != 0, Im[#] != 0] & [N[rad1, 10]],
54      And[Re[#] != 0, Im[#] != 0] & [N[rad2, 10]]];
55  problems = DeleteDuplicates[Cases[mod, f[_], {0, Infinity}]];
56  Print[problems];
57  Print[];
58  If[alllevels != True,
59    mod =
60      Simplify[
61        mod /. ({# -> Replace[#, f[val_] := func[val]]} & /@ problems)],
62    For[{i1 = 1}, MemberQ[problems, f[_]] == True, i1++,

```

```

63     i1 = Mod[i1, Length[problems], 1];
64     If[MatchQ[problems[[i1]], f[problem_]] /; FreeQ[problem, f[___]],
65       problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. f -> func);
66     For[{i2 = 1}, i2 <= Length[problems], i2++,
67       If[Head[problems[[i2]]] != Rule,
68         problems[[i2]] = problems[[i2]] /. problems[[i1]];
69       ]
70     ];
71   ];
72 ];
73 mod = Simplify[mod /. problems];
74 ];
75 Return[mod];
76 ]
77
78 ClearAll[denest11];
79 denest11[problem_, forcedmultipliers_ : 0] :=
80 Block[{radicand, outerpowers, reductionpower, extrapower,
81   multiplierstack, multipliers, multiplier, terms, history, attempt,
82   solution, updatehistory, stackchange, addmultipliers,
83   trymultiplier, newmultipliers, x, i1, i2, i3},
84 If[! MemberQ[problem,
85   Power[inner_?(Function[var, MemberQ[var, Power[a_, b_], Infinity]]],
86   outer_], {0, Infinity}],
87   Return[problem];
88 ];
89 Print["problem ", problem];
90 reductionpower = (LCM @@ (maxnestdepth[problem,
91   MatchQ[#, Power[_ , _Rational]] &] /. {Power[base_, exp_, _] :>
92   Denominator[exp]});
93 outerpowers = 1/reductionpower;
94 radicand = problem^reductionpower;
95 extrapower =
96 Times @@ ((LCM @@ #) & /@ (DeleteCases[
97   Cases[nestdepthsort[radicand,
98     MatchQ[#, Power[_ , _Rational]] &],
99     patt : {{expr_, nest_}, Repeated[{_, _}, {0, Infinity}]} /;
100     nest >= 2], {Except[Power[_ , _Rational]], _}, {2}] /.
101     patt : {Power[base_, exp_, nest_] :> Denominator[exp]});
102 Print["outer root: ", reductionpower, " extra root: ", extrapower];
103 Print[];
104
105 (*tries a multiplier
106 returns {<boolean progress>, {multiplier, minpoly, degree}, [answer]}*)
107 trymultiplier[radicand_, outerpowers_, multiplier_, best_,
108   worseknown_] /; MatchQ[outerpowers, Rational[1, _Integer]] :=
109 Block[{minpoly, degree, polygcd, possibilities, result, expected,
110   arg1},
111   minpoly = MinimalPolynomial[(multiplier*radicand)^outerpowers];
112   degree = Exponent[minpoly[arg1], arg1];
113   result = {False, {multiplier, minpoly, degree}};
114   Print["multiplier: ", multiplier, " minpoly degree: ", degree];
115   If[Or[degree < best, GCD[degree, best] < best],
116     polygcd =
117     PolynomialGCD[minpoly[arg1],
118       arg1^(1/outerpowers) - multiplier*radicand,
119     Extension -> Automatic];
120   If[And[Exponent[polygcd, arg1] < (1/outerpowers), polygcd != 1],
121     possibilities =
122     Flatten[Outer[Times,
123       arg1/(multiplier^outerpowers) /. Solve[polygcd == 0, arg1],
124       Exp[2*I*\[Pi]*Range[0, (1/outerpowers) - 1, 1]/(1/outerpowers)]]];
125   result =
126     DeleteDuplicates[
127     Select[possibilities, PossibleZeroQ[# - radicand^outerpowers] &]];
128   If[Length[result] > 1,
129     Print[Length[result], " possible solutions, taking the first"];
130   Print[result];

```

```

131     result = Take[result, 1];
132 ];
133 (*progress because a denesting occurred*)
134 result = {True, {multiplier, minpoly, degree}, result[[1]]},
135 If[polygcd != 1,
136   If[
137     Or[degree < best,
138       And[GCD[degree, best] < best, worseknown != True]],
139     (*progress because degree of minpoly has decreased*)
140     result = {True, {multiplier, minpoly, degree}}
141   ],
142   Print[
143     "PolynomialGCD error: ", {radicand, outterpower, multiplier,
144       minpoly}]
145   ],
146 ],
147 If[And[worseknown != True, degree != best],
148   (*progress because this multiplier has greater degree than a pre\
149 vious multiplier*)
150   Print["previous progress identified"];
151   result = {True, {multiplier, minpoly, degree}};
152   ]
153 ];
154 Return[result];
155 ];
156
157 (*form for attempts is {multiplier, minpoly, degree},
158 attempt2 is Null if only one attempt is available*)
159 newmultipliers[smaller_, larger_, reductionpower_] :=
160 Block[{done, new, primes, multiplierprimes, minprimes, maxprimes,
161   primeratio, multipliercap, arg1},
162   minprimes = 5;
163   maxprimes = 14;
164   primeratio = 100;
165   multipliercap = 1000;
166
167   (*two possibilites: reduction in degree,
168   reduction only in GCD of degrees*)
169   If[Or[larger == Null,
170     GCD[smaller[[-1]], larger[[-1]]] ==
171     Min[smaller[[-1]], larger[[-1]]],
172     (*one degree is a multiple of the other, usual case*)
173     primes =
174     DeleteCases[
175       FactorInteger[Discriminant[smaller[[2]][arg1], arg1],
176         Automatic], {-1, 1}];
177     primes =
178     Sort[DeleteCases[primes,
179       factor : {prime_, _} /;
180       And[! PrimeQ[prime], (Print["composite factor: ", factor];
181         True)]]], #1[[1]] <= #2[[2]] &];
182     primes = Flatten[primes[[All, 1]]];
183     multiplierprimes = primes;
184     For[{i3 = Length[primes]},
185       And[i3 > minprimes,
186         multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] > primeratio],
187       i3--,
188       multiplierprimes[[i3]] = 0;
189     ];
190     multiplierprimes =
191     DeleteCases[multiplierprimes, 0][[
192       1 ;; Min[
193         Max[minprimes, Ceiling[Log[multipliercap]/Log[reductionpower]]],
194         Length[multiplierprimes]]]];
195     new =
196     smaller[[1]]*
197     Drop[Union[
198       Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)],

```

```

199     primes], 1];
200     ,
201     (*different degree reductions in each degree from a larger degree\
202  *)
203     Print[
204     "GCD of smallest degrees is smaller than any observed degree: \
205 ", {smaller[{{1, -1}}], larger[{{1, -1}}]};
206     new = smaller[[1]]*larger[[1]]*{1};
207     Print[PolynomialGCD @@ {smaller[[2]][x], larger[[2]][x]};
208     ];
209     new = {{False, smaller}, ComplexitySort[new]};
210     Return[new];
211     ];
212
213     (*compute initial set of multipliers*)
214     If[ListQ[forcedmultipliers],
215     multipliers = {{False, {0, Infinity, Infinity}}, forcedmultipliers},
216     (*old, may be better*)
217     (*
218     terms=DeleteCases[Replace[#, {Times[a___,
219     Alternatives[_Rational, _Integer], b___] \[RuleDelayed] a*b,
220     Alternatives[_Rational, _Integer] \[Rule] 0, Complex[a_,
221     b_] \[RuleDelayed] Sign[b]*I}] & /@ (List @@ Expand[
222     RationalizeDenominator[radicand]]), 0];
223     *)
224     terms =
225     DeleteCases[
226     Replace[#, {Times[a___, Alternatives[_Rational, _Integer],
227     b___] :=> a*b, Alternatives[_Rational, _Integer] -> 0,
228     Complex[a_, b_] :=> Sign[b]*I}] & /@ (List @@
229     Expand[RationalizeDenominator[radicand]]), 0];
230     multipliers = Prepend[Replace[If[Head[#] === Times,
231
232     Times @@ (Function[{var},
233     var^(maxnestdepth[var,
234     Function[{var2},
235     MatchQ[var2, Power[_ , _Rational]]]] [[
236     1]] /. {{Power[base_,
237     exp_, _] :=> (Denominator[exp] - Numerator[exp])/
238     Numerator[exp], {Complex[
239     0, _], _] :=> -1}}] /@ (List @@ #)),
240     #^(maxnestdepth[#],
241     Function[{var}, MatchQ[var, Power[_ , _Rational]]]] [[
242     1]] /. {{Power[base_,
243     exp_, _] :=> (Denominator[exp] - Numerator[exp])/
244     Numerator[exp], {Complex[0, _], _] :=> -1}}
245     ],
246     Times[a___, Alternatives[_Rational, _Integer], b___] :=>
247     a*b] & /@ Sort[terms, (Re[#1] <= Re[#2]) &], 1];
248     multipliers = {{True, {0, Infinity, Infinity}}, multipliers};
249     ];
250
251     solution = problem;
252     history = {{0, Infinity, Infinity}};
253     (*multiplierstack form {{<do full set>,{<multiplier,minpoly,
254     degree for generating this set>},{<multipliers>}},...}*)
255     multiplierstack = {multipliers};
256     (*add power set of multipliers as second element in multiplierstack*)
257     (*multiplierstack=Append[multiplierstack,
258     {multiplierstack[[1,1]],ComplexitySort[DeleteCases[Union[(Times@@#)&/@
259     Subsets[DeleteCases[multipliers[[2]],1]]],mult_/;MemberQ[
260     multipliers[[2]],mult]]]}];*)
261     Catch[
262     For[{i1 = 1}, i1 <= Length[multiplierstack], i1++,
263     (*compute new multipliers from data if not already done*)
264     If[ListQ[multiplierstack[[i1, 2]]] != True,
265     multiplierstack[[i1]] =
266     multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);

```

```

267 ];
268 multipliers = multiplierstack[[i1]];
269 stackchange = False;
270 If[multipliers[[1, 2]] != history[[1]],
271   Print["multiplier count: ", Length[multipliers[[2]]], " from: ",
272     multipliers[[1, 2, {1, -1}]]],
273   If[! ListQ[forcedmultipliers],
274     Print["multiplier count: ", Length[multipliers[[2]]],
275       " multipliers: ", multipliers[[2]]],
276     Print["multiplier count: ", Length[multipliers[[2]]]];
277   ]
278 ];
279 For[{i2 = 1}, i2 <= Length[multipliers[[2]]], i2++,
280   multiplier = multipliers[[2, i2]];
281   updatehistory = False;
282   addmultipliers = False;
283   (*try a multiplier, different based on history*)
284   If[And[Length[history] >= 3, history[[-2]] != history[[1]],
285     attempt =
286       trymultiplier[radicand, outpower, multiplier,
287         history[[-1, -1]], True],
288     attempt =
289       trymultiplier[radicand, outpower, multiplier,
290         history[[-1, -1]], False];
291   ];
292   multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};
293
294   If[And[multipliers[[1, 1]] == True,
295     attempt[[2, -1]] <= history[[-1, -1]]],
296     addmultipliers = True
297   ];
298   (*check if progress has been made on this attempt,
299     no action taken if no progress*)
300   If[attempt[[1]] == True,
301     (*history will be updated*)
302     updatehistory = True;
303
304     (*decide if a stackchange will be made*)
305     If[history[[-1]] != history[[1]],
306       stackchange = True;
307     ];
308
309   If[ListQ[attempt[[-1]]] != True,
310     (*a solution was obtained*)
311     Print["success: ",
312       Join[attempt[[2, {1, -1}]],
313         If[IntegerQ[multiplier], {FactorInteger[multiplier]}, {}]]],
314     solution = Simplify[attempt[[-1]]];
315   If[
316     Or[history[[-1]] == history[[1]],
317       history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
318     (*not worth trying for additional denesting, exit immediately*)
319     history = Append[history, attempt[[2]]];
320     Throw["solution"],
321     (*further denesting will be attempted*)
322     Print["attempting additional denesting, current solution is:"];
323     Print[solution];
324   ];
325 ];
326
327 If[
328   And[history[[-1]] != history[[1]],
329     Or[attempt[[2, -1]] < history[[-1, -1]],
330       history[[-2]] == history[[1]]],
331   (*improvement has occurred, new multipliers will be created*)
332   Print["improvement ", attempt[[2, {1, -1}]]];
333   addmultipliers = True;
334 ];

```

```

335 ];
336
337 (*add new multipliers if needed*)
338 If[addmultipliers === True,
339   Block[{smaller, larger, done, new},
340     If[attempt[[2, -1]] <= history[[-1, -1]],
341       smaller = attempt[[2]];
342       larger = history[[-1]],
343       smaller = history[[-1]];
344       larger = attempt[[2]];
345     ];
346     new = {{False, smaller},
347           newmultipliers, {smaller,
348             If[larger != history[[1]], larger, Null], reductionpower}};
349
350   Print["adding new multipliers from: ", multiplier];
351   (*update multiplierstack with new multipliers*)
352   If[multipliers[[1, 1]] === False,
353     (*use new multipliers immediately*)
354     done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
355     multiplierstack = Insert[multiplierstack, new, i1 + 1];
356     If[i2 < Length[multipliers[[2]]],
357
358       multiplierstack =
359         Insert[multiplierstack, {multipliers[[1]],
360           multipliers[[2, i2 + 1 ;; -1]]}, i1 + 2];
361     ];
362     multiplierstack[[i1]] = done,
363     (*use new multipliers later*)
364     stackchange = False;
365     For[{i3 = Length[multiplierstack]}, i3 >= i1, i3--,
366
367       If[Or[multiplierstack[[i3, 1, 2]] === history[[1]],
368         new[[1, 2, -1]] > multiplierstack[[i3, 1, 2, -1]],
369         And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
370         LeafCount[new[[1, 2, 1]]] >=
371         LeafCount[multiplierstack[[i3, 1, 2, 1]]]],
372       multiplierstack = Insert[multiplierstack, new, i3 + 1];
373       Break[];
374     ];
375   ];
376 ];
377 ];
378 ];
379
380 (*update history if needed*)
381 If[updatehistory === True,
382   For[{i3 = Length[history]}, i3 >= 1, i3--,
383     If[history[[i3, -1]] > attempt[[2, -1]],
384       history = Insert[history, attempt[[2]], i3 + 1];
385       Break[];
386     ];
387   ];
388 ];
389
390 (*break inner loop if stack has changed*)
391 If[stackchange === True,
392   Break[];
393 ];
394 ];(*end of inner loop*)
395
396 (*add additional multipliers after first pass (if no forced multipliers)*)
397 If[And[i1 === 1, ! ListQ[forcedmultipliers]],
398   Block[{best, worst, extra, f},
399     best = Min[multiplierstack[[i1, 2, All, 2]]];
400     worst = Max[multiplierstack[[i1, 2, All, 2]]];
401     extra =

```

```

403 DeleteCases[
404   DeleteDuplicates[
405     Replace[(Distribute[
406       f @@ (#^Range[0, 1] & /@ (Cases[
407         multiplierstack[[i1, 2]], {mult_, degree_} /;
408         degree < worst][[All, 1]])),
409       List] /. {f[a___, Power[base_Times, exp_], b___] :=>
410         f[a, b, Sequence @@ ((List @@ base)^exp)],
411       f[a___, prod_Times, b___] :=> f[a, b, Sequence @@ prod],
412       f[a___, Power[base_, exp_], b___] :=>
413         f[a, b, (-1)^exp, (-base)^exp] /;
414         And[base != -1, Re[base] < 0],
415       f[a___, Power[base_, Rational[num1_, denom_]], b___,
416         Power[base_, Rational[num2_, denom_]], c___] :=>
417         f[a, b, c, base^(Mod[num1 + num2, denom]/denom)]] /.
418       f -> Times, Times[a___, _Integer, b___] :=> a*b, {1}] /.
419       Power[base_, Rational[a_, b_]] :=> base^(Mod[a, b]/b)],
420     Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
421   If[Length[extra] >= 1,
422     Print["adding multipliers based on progress in initial guess"];
423     multiplierstack =
424       Insert[multiplierstack, {multiplierstack[[i1, 1]], extra},
425         i1 + 1];
426   ];
427 ]
428 ];
429
430 ](*end of outer loop*),
431 "solution"
432 ];(*end of Catch, used to break the loop when a solution is found*)
433
434 Print["history: ", #[[{1, -1}]] & /@ history];
435 If[solution != problem,
436   Print[solution],
437   Print["no progress"];
438 ];
439 Print[];
440 Return[solution];
441 ]
442
443 ClearAll[RationalizeDenominator];
444 RationalizeDenominator[expr_] :=
445   Block[{conjugate, denom, x},
446     denom = Denominator[expr];
447     conjugate = MinimalPolynomial[denom];
448     Expand[
449       Numerator[expr]*PolynomialQuotient[conjugate[x], x - denom, x] /.
450       x -> 0]/(-1*conjugate[0])
451   ]
452
453 ClearAll[AlgebraicQ];
454 AlgebraicQ[num_] := Element[num, Algebraics]
455
456 ClearAll[nestdepths]
457 nestdepths[expr_, func_] :=
458   Block[{inner},
459     If[Depth[expr] === 1,
460       {expr, If[func[expr], 1, 0]},
461       inner = Map[nestdepths[#, func] &, (List @@ expr), {1}];
462       {expr,
463         Sequence @@
464         inner, ((Max @@ ((#[[-1]]) & /@ inner)) + If[func[expr], 1, 0])}
465     ]
466   ]
467
468 ClearAll[nestdepthsort]
469 nestdepthsort[expr_, func_] :=
470   Block[{data},

```



```

471 data = nestdepths[expr, func];
472 data =
473   data //.
474   patt : {a : Except[_List], b : __, c : Except[_List]} :> {{a, c},
475     b};
476 data = data //. patt : {rep : Repeated[{_, _}]} :> Sequence @@ patt;
477 data = Gather[{data}, (#1[[2]] == #2[[2]]) &];
478 data = Sort[data, (#1[[1, 2]] >= #2[[1, 2]]) &];
479 data
480 ]
481
482 ClearAll[maxnestdepth]
483 maxnestdepth[expr_, func_] :=
484   Block[{data, max},
485     data = nestdepths[{expr}, func][[2]];
486     max = data[[-1]];
487     data =
488       FixedPoint[
489         DeleteCases[# /. {a_, b_} :> {} /; b < max, {}, Infinity] &, data];
490     data =
491       If[And[ListQ[#], Length[#] > 2], Sequence @@ (#[[2 ;; -2]]), #] & /@
492         data;
493     If[Length[data] > 2,
494       data = data[[2 ;; -2]],
495       data = {data}
496     ];
497     data
498   ]
499
500 ClearAll[ComplexitySort];
501 ComplexitySort[expr_] :=
502   Return[Sort[expr,
503     Switch[LeafCount[#1] - LeafCount[#2], a_ /; a < 0, True,
504       a_ /; a > 0, False, 0, Abs[#1] <= Abs[#2]] &]];
505
506 ClearAll[Factorc]
507 Factorc[expr_] :=
508   Flatten[
509     fullfactor[
510       expr] //. {pow : {a : Except[_List], b : Except[_List]} :> {a^b},
511       prod : {rep : (Repeated[{Except[_List]}, {2,
512         Infinity}])} :> {Times[rep]},
513       sum : {rep : Repeated[{Except[_List]}, {2, Infinity}]} :>
514         Plus[rep],
515       pow2 : {{a : Except[_List]}, b : Except[_List]} :> {a^b},
516       pow3 : {{a : Except[_List]}, b : Except[_List]} :> {a^b}}][[1]]
517
518 ClearAll[fullfactor]
519 fullfactor[expr_] :=
520   Block[{factorlist, i = 0, j = 0, done = False},
521     factorlist = FactorList[expr];
522     While[! done && j < 5,
523       j++;
524       done = True;
525       For[{i = 1}, i <= Length[factorlist], i++,
526         Which[
527           Head[factorlist[[i, 1]]] === Power,
528           done = False;
529           factorlist[[
530             i]] = {{factorlist[[i, 1, 1]],
531               factorlist[[i, 1, 2]]*factorlist[[i, 2]]},
532           And[
533             MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]], !
534             PrimeQ[factorlist[[i, 1]], Abs[factorlist[[i, 1]]] != 1],
535           done = False;
536           factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@
537             FactorInteger[factorlist[[i, 1]]],
538           Head[factorlist[[i, 1]]] === Plus,

```

```

539     done = False;
540     factorlist[[i]] =
541       combine[fullfactor /@
542         Replace[factorlist[[i, 1]], Plus[a_, b_] :> {a, b}],
543       factorlist[[i, 2]]],
544     True,
545     factorlist[[i]] = {factorlist[[i]]}
546   ]
547 ];
548 factorlist = Flatten[factorlist, 1]
549 ];
550 factorlist
551 ]
552
553 ClearAll[combine];
554 combine[terms_List, pow_] :=
555 Block[{simpler = terms, base, bases, positions, factor, i, j, k},
556 For[{i = 1}, i <= Length[terms], i++,
557   base = terms[[i]];
558   For[{j = 1}, j < Length[base], j++,
559     For[{k = j + 1}, k <= Length[base], k++,
560       If[base[[j, 1]] == base[[k, 1]],
561         base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
562       base[[k]] = {0, 0}
563     ]
564   ];
565   If[base[[j, 1]] == 1,
566     base[[j]] = {0, 0}
567   ]
568 ];
569   simpler[[i]] = Delete[base, Position[base, {0, 0}]];
570 ];
571 bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
572 For[{i = 1}, i <= Length[bases], i++,
573   positions =
574     Position[#, {bases[[i]], Except[_List]}, 1] & /@ simpler;
575   factor =
576     Min[Extract[simpler[[#]], positions[[#]]][[1, 2]] & /@
577     Range[1, Length[simpler], 1]];
578   For[{j = 1}, j <= Length[simpler], j++,
579     simpler[[j, Sequence @@ (positions[[j, 1])], 2]] -= factor
580   ];
581   bases[[i]] = {bases[[i]], factor*pow};
582 ];
583 Append[bases, {simpler, pow}]
584 ]

```

B The corrected source

The file `src/corrected/StradFixed.wl`. Load it with `Get["StradFixed.wl"]`; the public symbols live in the context `RadicalDenest``.

```

1  (* ::Package:: *)
2
3  (* StradFixed.wl -- a corrected and hardened version of src/original/Strad.wl
4
5      The architecture of the original (wrapper -> marker -> multiplier search ->
6      minimal polynomial / polynomial GCD -> roots-of-unity orbit) is retained.
7      The changes are confined to correctness, robustness and resource control:
8
9      * every accepted candidate is certified, with exact algebra, to equal the
10     ORIGINAL problem (not the principal root of its power);
11     * the custom factorizer only applies the power identities
12     (ab)c -> abc and (F S)p -> Fp Sp when they are provably valid,
13     and its output is certified anyway;
14     * factoring and marking are restricted to exact numeric algebraic
15     subexpressions, so symbolic hosts are left untouched;
16     * the prime comparator, the stale-length slice, the Ceiling[Log] cap and
17     the {}[[1]] access are fixed; the multiplier cap is a real cap;
18     * the search has a trial budget, a time budget and a visited set;
19     * private context symbols replace the marker 'f' and the variable 'x';
20     * no Unprotect[Print]; diagnostics go through a "Verbose" option;
21     * a result is only returned if it is certified equal to the input AND
22     strictly cheaper under an explicit radical-complexity cost.
23
24     Author: analysis session of 5 September 2026. *)
25
26 BeginPackage["RadicalDenest`"];
27
28 Strad::usage = "Strad[expr, opts] denests exact algebraic radicals occurring in expr and returns an
29     expression certified equal to expr. Strad[expr, True] processes all nesting levels.";
30 DenestRadicals::usage = "DenestRadicals[expr, opts] is the option-driven wrapper behind Strad.
31     DenestRadicals[expr, allLevels] is the legacy positional form.";
32 DenestCore::usage = "DenestCore[problem, opts] runs the multiplier search on a single exact nested
33     radical and returns a certified equal expression (or problem itself).";
34 RationalizeDenominator::usage = "RationalizeDenominator[expr] rewrites expr so that its denominator is
35     rational, using the minimal polynomial of the denominator.";
36 RadicalDepth::usage = "RadicalDepth[expr] is the maximal number of nested rational-power nodes on a path
37     of expr.";
38 RadicalCost::usage = "RadicalCost[expr] is the lexicographic cost {#Root objects, radical depth, #
39     radical nodes, leaf count} used to decide whether a rewrite is an improvement.";
40 ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] is True when e is an exact numeric algebraic number.";
41 CertifiedEqualQ::usage = "CertifiedEqualQ[a, b] is True only when a - b is proved to be zero by exact
42     algebra.";
43 Factorc::usage = "Factorc[expr] is the branch-safe version of the original custom factorizer.";
44
45 Options[Strad] = Options[DenestRadicals] = {
46     "AllLevels" -> False, "Verbose" -> False, "Multipliers" -> Automatic,
47     "MaxTrials" -> 400, "TimeBudget" -> 120, "MultiplierCap" -> 1000,
48     "CertifyTime" -> 20, "Solver" -> Automatic, "Factor" -> True};
49 Options[DenestCore] = {
50     "Verbose" -> False, "Multipliers" -> Automatic, "MaxTrials" -> 400,
51     "TimeBudget" -> 120, "MultiplierCap" -> 1000, "CertifyTime" -> 20};
52
53 Begin["Private`"];
54
55 (* ----- *)
56 (* logging, predicates, certification *)
57 (* ----- *)
58
59 $verbose = False;
60 $certifyTime = 20;
61 log[args___] := If[TrueQ[$verbose], Print[args]];
62

```

```

56 ExactAlgebraicQ[e_] := TrueQ[NumericQ[e] && Precision[e] === Infinity &&
57   Quiet[Check[Element[e, Algebras], False]]];
58
59 exactRealQ[e_] := ExactAlgebraicQ[e] && TrueQ[Quiet[Element[e, Reals]]];
60 positiveRealQ[e_] := exactRealQ[e] && TrueQ[Quiet[e > 0]];
61
62 (* numeric prefilter (cheap rejection) followed by exact certification *)
63 CertifiedEqualQ[a_, b_] := Module[{d, num, r},
64   If[! ExactAlgebraicQ[a] || ! ExactAlgebraicQ[b], Return[False]];
65   d = a - b;
66   num = Quiet[N[d, 30]];
67   If[NumberQ[num] && Abs[num] > 10^-20, Return[False]];
68   r = Quiet[TimeConstrained[Check[RootReduce[d] == 0, $Failed], $certifyTime, $Failed]];
69   If[r === True, Return[True]];
70   If[r === False, Return[False]];
71   r = Quiet[TimeConstrained[Check[PossibleZeroQ[d, Method -> "ExactAlgebras"], $Failed], $certifyTime,
72     $Failed]];
73   TrueQ[r]];
74
75 (* ----- *)
76 (* representation cost *)
77
78 RadicalDepth[e_] := If[AtomQ[e], 0,
79   If[MatchQ[e, Power[_ , _Rational]], 1 + RadicalDepth[First[e]],
80     Max[Prepend[RadicalDepth /@ (List @@ e), 0]]];
81
82 RadicalCost[e_] := {Count[e, _Root | _AlgebraicNumber, {0, Infinity}],
83   RadicalDepth[e], Count[e, Power[_ , _Rational], {0, Infinity}], Count[e, _?AtomQ, {-1}], LeafCount[e
84   ]];
85
86 cheaperQ[new_, old_] := Order[RadicalCost[new], RadicalCost[old]] === 1;
87 pickCheapest[list_List] := First[SortBy[list, RadicalCost]];
88
89 safeSimplify[e_] := Quiet[TimeConstrained[Simplify[e], $certifyTime, e]];
90
91 (* ----- *)
92 (* denominator rationalization (valid for nonzero algebraic denominators) *)
93
94 RationalizeDenominator[expr_] := Module[{x, e, num, denom, minpoly, q},
95   e = Together[expr];
96   denom = Denominator[e]; num = Numerator[e];
97   If[denom === 1 || ! ExactAlgebraicQ[denom], Return[expr]];
98   minpoly = Quiet[Check[MinimalPolynomial[denom, x], $Failed]];
99   If[! PolynomialQ[minpoly, x], Return[expr]];
100   q = PolynomialQuotient[minpoly, x - denom, x] /. x -> 0;
101   Expand[num*q]/(-minpoly /. x -> 0)];
102
103 (* ----- *)
104 (* structural helpers *)
105
106
107 radicalQ[e_] := MatchQ[e, Power[_ , _Rational]];
108 nestedRadicalQ[e_] := RadicalDepth[e] >= 2;
109
110 (* LCM of exponent denominators of the rational-power nodes of maximal depth *)
111 reductionExponent[e_] := Module[{d = RadicalDepth[e], nodes},
112   nodes = Select[Cases[e, Power[_ , _Rational], {0, Infinity}], RadicalDepth[#] == d &];
113   If[nodes === {}, 1, LCM @@ (Denominator[Last[#]] & /@ nodes)];
114
115 summands[e_] := If[Head[e] === Plus, List @@ e, {e}];
116 factorsOf[e_] := If[Head[e] === Times, List @@ e, {e}];
117
118 complexitySort[list_] := SortBy[list, {LeafCount[#], Quiet[N[Abs[#], 20]]} &];
119
120 (* ----- *)
121 (* branch-safe custom factorizer *)

```

```

122 (* ----- *)
123
124 (* (a^b)^c -> a^(b c) is valid when c is an integer or b is real with -1 < b <= 1 *)
125 powerMergeSafeQ[b_, c_] := IntegerQ[c] || (exactRealQ[b] && TrueQ[-1 < b <= 1]);
126
127 Factorc[expr_] := Module[{r},
128   r = Quiet[Check[factorcRaw[expr], expr]];
129   If[ExactAlgebraicQ[expr], If[CertifiedEqualQ[r, expr], r, expr], r];
130
131 factorcRaw[expr_] :=
132   Flatten[fullfactor[expr] //. {
133     pow : {a : Except[_List], b : Except[_List]} :> {a^b},
134     prod : {rep : (Repeated[{Except[_List]}, {2, Infinity}])} :> {Times[rep]},
135     sum : {rep : Repeated[{Except[_List]}, {2, Infinity}]} :> Plus[rep],
136     pow2 : {{a : Except[_List]}, b : Except[_List]} :> {a^b},
137     pow3 : {{a : Except[_List]}, b : Except[_List]} :> {a^b}}[[1]];
138
139 fullfactor[expr_] := Module[{factorlist, i = 0, j = 0, done = False},
140   factorlist = FactorList[expr];
141   While[! done && j < 5,
142     j++;
143     done = True;
144     For[i = 1, i <= Length[factorlist], i++,
145       Which[
146         Head[factorlist[[i, 1]]] === Power && powerMergeSafeQ[factorlist[[i, 1, 2]], factorlist[[i, 2]]],
147         done = False;
148         factorlist[[i]] = {{factorlist[[i, 1, 1]], factorlist[[i, 1, 2]]*factorlist[[i, 2]]},
149         And[MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]]], ! PrimeQ[factorlist[[i, 1]]], Abs[
150         factorlist[[i, 1]]] != 1],
151         done = False;
152         factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@ FactorInteger[factorlist[[i, 1]]],
153         Head[factorlist[[i, 1]]] === Plus,
154         done = False;
155         factorlist[[i]] = combine[fullfactor /@ (List @@ factorlist[[i, 1]]), factorlist[[i, 2]],
156         True,
157         factorlist[[i]] = {factorlist[[i]]}];
158     factorlist = Flatten[factorlist, 1];
159   ]
160
161 (* common-factor extraction from a sum raised to the power pow:
162   (F S)^pow -> F^pow S^pow only when pow is an integer, or F > 0, or S > 0 *)
162 combine[terms_List, pow_] := Module[{simpler = terms, base, bases, positions, factor, i, j, k, extracted
163   , remaining, fval, sval},
164   For[i = 1, i <= Length[terms], i++,
165     base = terms[[i]];
166     For[j = 1, j <= Length[base], j++,
167       For[k = j + 1, k <= Length[base], k++,
168         If[base[[j, 1]] === base[[k, 1]] && base[[j]] != {0, 0},
169           base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
170         base[[k]] = {0, 0}];
171     If[base[[j, 1]] === 1, base[[j]] = {0, 0}];
172     simpler[[i]] = DeleteCases[base, {0, 0}];
173     bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
174     If[bases === {}, Return[{{simpler, pow}}];
175     extracted = simpler;
176     bases = Table[
177       positions = Position[#, {bases[[i]], Except[_List]}, 1] & /@ extracted;
178       factor = Min[Extract[extracted[[#]], positions[[#]][[1, 2]]] & /@ Range[Length[extracted]]];
179       For[j = 1, j <= Length[extracted], j++,
180         extracted[[j, Sequence @@ (positions[[j, 1]]), 2]] -= factor];
181       {bases[[i]], factor},
182       {i, Length[bases]};
183   fval = Times @@ (#[[1]]^#[[2]] & /@ bases);
184   sval = Plus @@ ((Times @@ (#[[1]]^#[[2]] & /@ #)) & /@ extracted);
185   If[IntegerQ[pow] || positiveRealQ[fval] || positiveRealQ[sval],
186     Append[({#[[1]], #[[2]]*pow} & /@ bases), {extracted, pow}],
187     {{simpler, pow}}];

```

```

188 (* apply the factorizer only to maximal exact numeric non-atomic subexpressions *)
189 numericTargetQ[s_] := ! AtomQ[s] && ExactAlgebraicQ[s];
190 safeFactorAll[expr_] := If[numericTargetQ[expr], Factorc[expr], expr /. s_?numericTargetQ -> Factorc[s
    ]];
191
192 (* ----- *)
193 (* the wrapper *)
194 (* ----- *)
195
196 Strad[expr_, opts : OptionsPattern[]] := DenestRadicals[expr, opts];
197 Strad[expr_, all : (True | False), opts : OptionsPattern[]] := DenestRadicals[expr, "AllLevels" -> all,
    opts];
198 Strad[l_List, rest___] := Strad[#, rest] & /@ l;
199
200 DenestRadicals[expr_, all : (True | False), opts : OptionsPattern[]] := DenestRadicals[expr, "AllLevels"
    -> all, opts];
201 DenestRadicals[l_List, rest___] := DenestRadicals[#, rest] & /@ l;
202
203 sumQ[expression_] := MemberQ[expression, _Plus | Complex[Except[0], _], {0, Infinity}];
204
205 DenestRadicals[expr_, opts : OptionsPattern[]] :=
206   Block[{$verbose = OptionValue["Verbose"], $certifyTime = OptionValue["CertifyTime"]},
207     Module[{level, mod, problems, solver, coreOpts, rules, result, candidates, i1, i2, iter, best},
208       If[Precision[expr] != Infinity && ! FreeQ[expr, _Real | _Complex?InexactNumberQ], Return[expr]];
209       level = If[TrueQ[OptionValue["AllLevels"]], ReplaceRepeated, ReplaceAll];
210       solver = OptionValue["Solver"];
211       coreOpts = FilterRules[{opts}, Options[DenestCore]];
212       If[solver === Automatic, solver = recursiveCore[#, coreOpts] &];
213       (* Root/AlgebraicNumber objects are first converted to radicals when possible *)
214       mod = expr;
215       If[! FreeQ[mod, _Root | _AlgebraicNumber],
216         mod = Quiet[TimeConstrained[ToRadicals[mod], $certifyTime, mod]];
217       If[! FreeQ[mod, _Root | _AlgebraicNumber], mod = expr]];
218       mod = If[TrueQ[OptionValue["Factor"]], safeFactorAll[mod], mod];
219       (* mark exact algebraic radicands raised to rational powers *)
220       mod = level[mod, radicand?ExactAlgebraicQ^pow_Rational -> normalizeRadicand[radicand, pow]];
221       (* group products of two markers whose radicands both contain radicals *)
222       mod = mod /. Times[a___, mark[rad1_, pow1_], b___, mark[rad2_, pow2_], c___] ->
223         Times[a, b, c, mark[(rad1^pow1)*(rad2^pow2)]]; And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
224       mod = level[mod, mark[rad_, pow_] -> mark[rad^pow]];
225       mod = mod /. Times[a___, mark[rad1_], b___, mark[rad2_], c___] -> Times[a, b, c, mark[rad1*rad2]]
226       /;
227       And[And[Re[#] != 0, Im[#] != 0] &[N[rad1, 10]], And[Re[#] != 0, Im[#] != 0] &[N[rad2, 10]]];
228       problems = DeleteDuplicates[Cases[mod, mark[_], {0, Infinity}]];
229       log["problems: ", problems];
230       If[! TrueQ[OptionValue["AllLevels"]],
231         rules = (# -> Replace[#, mark[val_] -> solver[val]]) & /@ problems;
232         result = mod /. rules,
233         (* all levels: resolve innermost markers first, with an iteration cap *)
234         iter = 0;
235         For[i1 = 1, MemberQ[problems, mark[___]] && iter < 20 Length[problems] + 20, i1++,
236           iter++;
237           i1 = Mod[i1, Length[problems], 1];
238           If[MatchQ[problems[[i1]], mark[problem___] /; FreeQ[problem, mark[___]]],
239             problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. mark -> solver);
240             For[i2 = 1, i2 <= Length[problems], i2++,
241               If[Head[problems[[i2]]] != Rule, problems[[i2]] = problems[[i2]] /. problems[[i1]]];];
242             result = mod /. Cases[problems, _Rule];
243             result = result /. mark[v_] -> v];
244         (* choose the cheapest certified representative among input, result, Simplify[result] *)
245         candidates = DeleteDuplicates[{result, safeSimplify[result], denestInnerSurds[result]}];
246         If[ExactAlgebraicQ[expr],
247           candidates = Select[candidates, CertifiedEqualQ[#, expr] &];
248           candidates = Select[candidates, cheaperQ[#, expr] &];
249           If[candidates === {}, expr, pickCheapest[candidates]]];
250       (* the core, followed by one further all-level pass on a partial solution *)
251       $recursion = 0;

```

```

252 recursiveCore[problem_, coreOpts_] := Module[{sol, again},
253   sol = DenestCore[problem, Sequence @@ coreOpts];
254   If[sol != problem && RadicalDepth[sol] >= 2 && $recursion < 2,
255     again = Block[{$recursion = $recursion + 1},
256       DenestRadicals[sol, "AllLevels" -> True, Sequence @@ coreOpts]];
257     If[cheaperQ[again, sol] && CertifiedEqualQ[again, problem], sol = again];
258   sol];
259
260 (* value-preserving normalization of a radicand (the original marker rule), certified *)
261 normalizeRadicand[radicand_, pow_] := Module[{factored, i, j, value},
262   value = Quiet[Check[
263     factored = FactorTermsList[radicand];
264     factored[[2]] = Flatten[{Replace[Factorrc[factored[[2]]], prod_Times -> List @@ prod]}];
265     factored[[1]] = DeleteCases[#, ComplexExpand[factored[[1]]/#] & GCD[Re[factored[[1]]], Im[
266       factored[[1]]]], 1];
267     factored[[1]] = #[[1]]^#[[2]] & /@ Flatten[FactorInteger /@ factored[[1]], 1];
268     factored = Flatten[factored];
269     factored = factored /. {a___, expr1_, b___, expr2_, c___} -> {a, b, c, ComplexExpand[expr1*expr2
270     ]} /; And[sumQ[expr1], sumQ[expr2]];
271     j = 1;
272     For[i = 1, i <= Length[factored], i++,
273       If[TrueQ[NRe[factored[[i]]], 100] < 0, factored[[i]] = -factored[[i]]; j *= -1];
274       factored = DeleteCases[Prepend[factored, j], 1];
275       safeSimplify[Times @@ (safeSimplify[RationalizeDenominator[Together[#]]] & /@ factored)],
276       $Failed];
277   If[value === $Failed || ! CertifiedEqualQ[value, radicand], value = radicand];
278   mark[value, pow]];
279
280 (* ----- *)
281 (* quadratic surds: Sqrt[a + b Sqrt[c]] with rational a, b, c *)
282 (* If a > 0 and a^2 - b^2 c = delta^2 with delta rational, then *)
283 (* Sqrt[a + b Sqrt[c]] = Sqrt[(a+delta)/2] + Sign[b] Sqrt[(a-delta)/2]; *)
284 (* the right-hand side is nonnegative, so it is the principal root. *)
285 (* ----- *)
286
287 rationalQ[e_] := MatchQ[e, _Integer | _Rational];
288 (* named patterns are essential: two identical '_?rationalQ' factors would be
289 combined by Times into (_?rationalQ)^(3/2) before matching *)
290 surdParts[rad_] := Replace[Expand[rad], {
291   a_?rationalQ + b_?rationalQ Sqrt[c_?rationalQ] -> {a, b, c},
292   a_?rationalQ + Sqrt[c_?rationalQ] -> {a, 1, c},
293   _ -> $Failed};
294
295 denestSurd[a_, b_, c_] := Module[{delta, s, u, v, r},
296   If[a == 0 || b == 0 || c <= 0, Return[$Failed]];
297   If[a < 0,
298     If[TrueQ[a + b Sqrt[c] < 0], r = denestSurd[-a, -b, c]; Return[If[r === $Failed, $Failed, I r]],
299     Return[$Failed]];
300   delta = a^2 - b^2 c;
301   If[delta < 0, Return[$Failed]];
302   s = Sqrt[delta];
303   If[! rationalQ[s], Return[$Failed]];
304   u = (a + s)/2; v = (a - s)/2;
305   Sqrt[u] + Sign[b] Sqrt[v];
306
307 denestSurdPower[rad_] := Module[{p = surdParts[rad], r},
308   If[p === $Failed, Return[Sqrt[rad]]];
309   r = denestSurd @@ p;
310   If[r === $Failed, Sqrt[rad], r];
311
312 denestInnerSurds[e_] := FixedPoint[Replace[#, Power[rad_, Rational[1, 2]] -> denestSurdPower[rad], {0,
313   Infinity}] &, e, 4];
314
315 (* variants of a certified candidate; all are value preserving *)
316 polish[cand_] := DeleteDuplicates[Flatten[{cand, safeSimplify[cand],
317   safeSimplify[Quiet[Check[RationalizeDenominator[Together[cand]], cand]]],
318   denestInnerSurds[cand], safeSimplify[denestInnerSurds[cand]]}]];
319
320 (* ----- *)
321 (* the core: multiplier search *)
322 (* ----- *)

```



```

316
317 DenestCore[problem_, OptionsPattern[]] :=
318   Block[{$verbose = OptionValue["Verbose"], $certifyTime = OptionValue["CertifyTime"],
319     $trials = 0, $maxTrials = OptionValue["MaxTrials"], $deadline = AbsoluteTime[] + OptionValue["
320       TimeBudget"],
321     $visited = <|>, $cap = OptionValue["MultiplierCap"]},
322   Module[{radicand, outpower, reductionpower, multiplierstack, multipliers, multiplier, terms,
323     history,
324     attempt, solution, updatehistory, stackchange, addmultipliers, forced, i1, i2, i3, r},
325     If[! ExactAlgebraicQ[problem] || ! nestedRadicalQ[problem], Return[problem]];
326     If[TrueQ[CertifiedEqualQ[problem, 0]], Return[0]];
327     reductionpower = reductionExponent[problem];
328     If[! IntegerQ[reductionpower] || reductionpower < 2, Return[problem]];
329     outpower = 1/reductionpower;
330     radicand = problem^reductionpower;
331     (* fast path for quadratic surds *)
332     If[MatchQ[problem, Power[_ , Rational[1, 2]]],
333       r = denestSurdPower[First[problem]];
334       If[r != problem && cheaperQ[r, problem] && CertifiedEqualQ[r, problem],
335         log["quadratic surd fast path: ", problem, " -> ", r]; Return[r];];
336     log["problem ", problem];
337     log["outer root: ", reductionpower];
338     forced = OptionValue["Multipliers"];
339     If[ListQ[forced],
340       multipliers = {{False, {0, Infinity, Infinity}}, forced},
341       terms = DeleteCases[Replace[#, {Times[a___, Alternatives[_Rational, _Integer], b___] :> a*b,
342         Alternatives[_Rational, _Integer] -> 0, Complex[a_, b_] :> Sign[b]*I}] & /@
343         summands[Expand[RationalizeDenominator[radicand]]], 0];
344       terms = SortBy[terms, Quiet[N[Re[#], 20]] &];
345       multipliers = Prepend[DeleteCases[complementaryMultiplier /@ terms, 1], 1];
346       multipliers = {{True, {0, Infinity, Infinity}}, DeleteDuplicates[multipliers]};
347     solution = problem;
348     history = {{0, Infinity, Infinity}};
349     multiplierstack = {multipliers};
350     r = Catch[
351       For[i1 = 1, i1 <= Length[multiplierstack], i1++,
352         If[ListQ[multiplierstack[[i1, 2]]] != True,
353           multiplierstack[[i1]] = multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);
354           multipliers = multiplierstack[[i1]];
355           stackchange = False;
356           If[multipliers[[1, 2]] != history[[1]],
357             log["multiplier count: ", Length[multipliers[[2]]], " from: ", multipliers[[1, 2, {1, -1}]],
358             log["multiplier count: ", Length[multipliers[[2]]], " multipliers: ", multipliers[[2]]];
359             For[i2 = 1, i2 <= Length[multipliers[[2]]], i2++,
360               multiplier = multipliers[[2, i2]];
361               updatehistory = False;
362               addmultipliers = False;
363               If[KeyExistsQ[$visited, multiplier],
364                 log["skipping visited multiplier ", multiplier];
365                 multiplierstack[[i1, 2, i2]] = {multiplier, $visited[multiplier]};
366                 Continue[];
367               If[$trials >= $maxTrials || AbsoluteTime[] > $deadline,
368                 log["budget exhausted after ", $trials, " trials"];
369                 Throw[Null, "budget"];
370                 $trials++;
371               attempt = If[And[Length[history] >= 3, history[[-2]] != history[[1]],
372                 tryMultiplier[problem, radicand, outpower, multiplier, history[[-1, -1]], True],
373                 tryMultiplier[problem, radicand, outpower, multiplier, history[[-1, -1]], False]];
374               $visited[multiplier] = attempt[[2, -1]];
375               multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};
376               If[And[multipliers[[1, 1]] == True, attempt[[2, -1]] <= history[[-1, -1]], addmultipliers =
377                 True];
378               If[attempt[[1]] == True,
379                 updatehistory = True;
380                 If[history[[-1]] != history[[1]], stackchange = True];
381                 If[Length[attempt] == 3,
382                   log["success: ", attempt[[2, {1, -1}]]];
383                   solution = If[cheaperQ[attempt[[-1]], solution], attempt[[-1]], solution];

```



```

381     If[Or[history[[-1]] === history[[1]], history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
382     history = Append[history, attempt[[2]]];
383     Throw[Null, "solution"],
384     log["attempting additional denesting, current solution is: ", solution]]];
385     If[And[history[[-1]] != history[[1]], Or[attempt[[2, -1]] < history[[-1, -1]], history[[-2]]
=== history[[1]]],
386     log["improvement ", attempt[[2, {1, -1}]]];
387     addmultipliers = True];
388     If[addmultipliers === True,
389     Module[{smaller, larger, done, new},
390     If[attempt[[2, -1]] <= history[[-1, -1]],
391     smaller = attempt[[2]]; larger = history[[-1]],
392     smaller = history[[-1]]; larger = attempt[[2]];
393     new = {{False, smaller}, newMultipliers, {smaller, If[larger != history[[1]], larger, Null],
reductionpower}};
394     log["adding new multipliers from: ", multiplier];
395     If[multipliers[[1, 1]] === False,
396     done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
397     multiplierstack = Insert[multiplierstack, new, i1 + 1];
398     If[i2 < Length[multipliers[[2]]],
399     multiplierstack = Insert[multiplierstack, {multipliers[[1]], multipliers[[2, i2 + 1 ;;
-1]]}, i1 + 2];
400     multiplierstack[[i1]] = done,
401     stackchange = False;
402     For[i3 = Length[multiplierstack], i3 >= i1, i3--,
403     If[Or[multiplierstack[[i3, 1, 2]] === history[[1]], new[[1, 2, -1]] > multiplierstack[[i3,
1, 2, -1]],
404     And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
405     LeafCount[new[[1, 2, 1]]] >= LeafCount[multiplierstack[[i3, 1, 2, 1]]]],
406     multiplierstack = Insert[multiplierstack, new, i3 + 1];
407     Break[]]]];
408     If[updatehistory === True,
409     For[i3 = Length[history], i3 >= 1, i3--,
410     If[history[[i3, -1]] > attempt[[2, -1]],
411     history = Insert[history, attempt[[2]], i3 + 1];
412     Break[]]]];
413     If[stackchange === True, Break[]];
414     (* products of promising initial multipliers, after the first pass *)
415     If[And[i1 === 1, ! ListQ[forced]],
416     Module[{worst, extra, ff, good},
417     good = Cases[multiplierstack[[i1, 2]], {_, _?NumericQ}];
418     If[Length[good] >= 2,
419     worst = Max[good][[All, 2]];
420     extra = DeleteCases[DeleteDuplicates[
421     Replace[(Distribute[ff @@ (#^Range[0, 1] & /@ (Cases[good, {mult_, degree_} /; degree <
worst][[All, 1]])), List] /. {
422     ff[a___, Power[base_Times, exp_], b___] :> ff[a, b, Sequence @@ ((List @@ base)^exp)
],
423     ff[a___, prod_Times, b___] :> ff[a, b, Sequence @@ prod],
424     ff[a___, Power[base_, exp_], b___] :> ff[a, b, (-1)^exp, (-base)^exp] /; And[base !=
-1, TrueQ[Re[base] < 0]],
425     ff[a___, Power[base_, Rational[num1_, denom_]], b___, Power[base_, Rational[num2_,
denom_]], c___] :>
426     ff[a, b, c, base^(Mod[num1 + num2, denom]/denom)]] /. ff -> Times,
427     Times[a___, _Integer, b___] :> a*b, {1}] /. Power[base_, Rational[a_, b_]] :> base^(
Mod[a, b]/b)],
428     Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
429     extra = Select[extra, ExactAlgebraicQ[#] && ! TrueQ[# == 0] &];
430     If[Length[extra] > $cap, extra = Take[complexitySort[extra], $cap]];
431     If[Length[extra] >= 1,
432     log["adding multipliers based on progress in initial guess"];
433     multiplierstack = Insert[multiplierstack, {multiplierstack[[i1, 1]], extra, i1 + 1}]]],
434     "solution" | "budget"];
435     log["history: ", #[[{1, -1}]] & /@ history];
436     log["trials: ", $trials];
437     If[solution != problem && CertifiedEqualQ[solution, problem], solution, problem]]];
438
439     (* complementary multiplier for one term of the expanded radicand *)

```

```

440 complementaryMultiplier[term_] := Times @@ (complementaryFactor /@ factorsOf[term]);
441 complementaryFactor[Power[b_, Rational[n_, d_]]] := b^((d - n)/d);
442 complementaryFactor[Complex[0, _]] := -I;
443 complementaryFactor[e_] := Module[{deep = Cases[e, Power[_ , _Rational], {0, Infinity}], maxd},
444   If[deep === {}, 1,
445     maxd = Max[RadicalDepth /@ deep];
446     Times @@ (complementaryFactor /@ Select[deep, RadicalDepth[#] == maxd &])];
447
448 (* one trial: {progress?, {multiplier, minpoly, degree}, [certified answer]} *)
449 tryMultiplier[problem_, radicand_, outerpower_, multiplier_, best_, worseknown_] :=
450 Module[{arg1, r = 1/outerpower, minpoly, degree, polygcd, possibilities, roots, certified, result},
451   result = {False, {multiplier, Null, Infinity}};
452   If[! ExactAlgebraicQ[multiplier] || TrueQ[Quiet[PossibleZeroQ[multiplier, Method -> "ExactAlgebraics"]]],
453     log["invalid multiplier skipped: ", multiplier]; Return[result];
454   minpoly = Quiet[Check[MinimalPolynomial[(multiplier*radicand)^outerpower, arg1], $Failed]];
455   If[! PolynomialQ[minpoly, arg1], log["minimal polynomial failed for ", multiplier]; Return[result];
456   degree = Exponent[minpoly, arg1];
457   result = {False, {multiplier, Function[Evaluate[minpoly /. arg1 -> #], degree]}};
458   log["multiplier: ", multiplier, " minpoly degree: ", degree];
459   If[Or[degree <= best, GCD[degree, best] < best],
460     polygcd = Quiet[Check[PolynomialGCD[minpoly, arg1^r - multiplier*radicand, Extension -> Automatic],
461       $Failed]];
462     If[! PolynomialQ[polygcd, arg1], log["PolynomialGCD failed for ", multiplier]; Return[result];
463     If[And[0 < Exponent[polygcd, arg1] < r],
464       roots = Quiet[Check[arg1 /. Solve[polygcd == 0, arg1], {}]];
465       roots = Select[Flatten[{roots}], ExactAlgebraicQ];
466       possibilities = Flatten[Outer[Times, roots/(multiplier^outerpower), Exp[2*I*Pi*Range[0, r - 1]/r]]];
467       certified = Select[possibilities, CertifiedEqualQ[#, problem] &];
468       If[certified != {},
469         certified = DeleteDuplicates[Flatten[polish /@ certified]];
470         result = {True, result[[2]], pickCheapest[certified]};
471         log["GCD of degree ", Exponent[polygcd, arg1], " but no certified candidate for ", multiplier];
472         If[Or[degree < best, And[GCD[degree, best] < best, worseknown != True]], result[[1]] = True];
473         If[polygcd != 1 && Exponent[polygcd, arg1] > 0,
474           If[Or[degree < best, And[GCD[degree, best] < best, worseknown != True]], result[[1]] = True];
475           log["PolynomialGCD returned a constant: ", {radicand, outerpower, multiplier}];
476           If[And[worseknown != True, degree != best],
477             log["previous progress identified"];
478             result[[1]] = True];
479           result];
480
481 (* generate a batch of multipliers from a discriminant, with a real cap *)
482 newMultipliers[smaller_, larger_, reductionpower_] :=
483 Module[{arg1, new, primes, multiplierprimes, minprimes = 5, primeratio = 100, count, t, i3, disc},
484   If[Or[larger === Null, GCD[smaller[[-1]], larger[[-1]]] == Min[smaller[[-1]], larger[[-1]]]],
485     disc = Quiet[Check[Discriminant[smaller[[2]]arg1, arg1], $Failed]];
486     If[! IntegerQ[disc] || disc == 0, Return[{{False, smaller}, {}}]];
487     primes = DeleteCases[FactorInteger[disc, Automatic], {-1, 1}];
488     primes = Select[primes, If[PrimeQ[#[[1]]], True, log["composite factor dropped: ", #]; False] &];
489     primes = SortBy[primes, First][[All, 1]];
490     multiplierprimes = primes;
491     For[i3 = Length[primes], And[i3 > minprimes, multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] >
492       primeratio], i3--,
493       multiplierprimes[[i3]] = 0];
494     multiplierprimes = DeleteCases[multiplierprimes, 0];
495     (* largest t with reductionpower^t - 1 <= cap, computed in integer arithmetic *)
496     t = 0; While[reductionpower^(t + 1) - 1 <= $cap, t++];
497     count = Min[Max[1, t], Length[multiplierprimes]];
498     multiplierprimes = Take[multiplierprimes, count];
499     new = smaller[[1]]*Drop[Union[Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)], primes],
500       1],
501     log["GCD of smallest degrees is smaller than any observed degree: ", {smaller[[{1, -1}]], larger
502       [[{1, -1}]]]];
503     new = {smaller[[1]]*larger[[1]]};
504     new = DeleteCases[new, m_ /; KeyExistsQ[$visited, m]];
505     {{False, smaller}, complexitySort[new]};

```

```
502  
503 End[];  
504 EndPackage[];
```

C Experiment harness

All scripts are in `src/code-review/unified/harness/`; the recorded outputs are in `src/code-review/unified/lo`. The runner used for the original program is listed below; `runner_fixed.wl` is identical except that it loads the corrected package. The case files (`exp_original.wl`, `exp_helpers.wl`, `exp_round2.wl`, `exp_round3.wl`, `exp_fixed.wl`), the randomized families (`exp_fuzz.wl`, `exp_fuzz_fixed.wl`), the table generator (`export_tables.wl`), the script that exercises the reviews' proposed kernels (`test_review_kernels.wl`) and the shell script that runs the reviews' suites (`run_suites.sh`) are self-contained apart from absolute paths at their top.

```

1  (* Generic experiment runner for Strad.wl analysis *)
2  $srcFile = "C:/RadicalDenest/.claude/worktrees/strad-denesting-analysis-6158e6/src/original/Strad.wl";
3  $printAttrsBefore = Attributes[Print];
4  Get[$srcFile];
5  $printAttrsAfter = Attributes[Print];
6
7  SetAttributes[capture, HoldAll];
8  capture[expr_] := Module[{msgs = {}, res},
9    res = Internal`HandlerBlock[{"Message",
10     Function[m, If[TrueQ[m[[2]]], AppendTo[msgs, ToString[Extract[m, {1, 1}, HoldForm]]]]], expr];
11    {res, DeleteDuplicates[msgs]};
12
13  radDepth[e_] := If[AtomQ[e], 0,
14    If[MatchQ[e, Power[_ , _Rational]], 1 + radDepth[e[[1]]],
15    Max[Prepend[radDepth /@ (List @@ e), 0]]];
16
17  SetAttributes[runCase, HoldAll];
18  runCase[id_, expr_, expected_, tlimit_: 120] := Module[
19    {t, res, msgs, out, numQ, numOK, exactOK, inputVal, depthIn, depthOut},
20    inputVal = ReleaseHold[Hold[expr] /. {HoldPattern[Strad[e_, ___]] :> e, HoldPattern[DenestRadicals3[e_, ___]] :> e, HoldPattern[denest11[e_, ___]] :> e}];
21    {t, {res, msgs}} = AbsoluteTiming[TimeConstrained[capture[expr], tlimit, {$TimedOut, {"TIMEOUT"}}]];
22    numQ = NumericQ[inputVal] && NumericQ[res];
23    numOK = If[numQ, Quiet[TrueQ[Abs[N[res - inputVal, 40]] < 10-30], "n/a"];
24    exactOK = If[expected === None, "n/a",
25      Quiet[TimeConstrained[TrueQ[RootReduce[res - expected] === 0], 30, "rr-timeout"]];
26    depthIn = radDepth[inputVal]; depthOut = radDepth[res];
27    out = <|"id" -> id, "input" -> ToString[HoldForm[expr], InputForm],
28      "result" -> ToString[res, InputForm], "time" -> Round[t, 0.001],
29      "numeric" -> numOK, "exact" -> exactOK, "depth" -> {depthIn, depthOut},
30      "messages" -> msgs|>;
31    Print[id, " | ", ToString[HoldForm[expr], InputForm], "\n -> ", ToString[res, InputForm],
32      "\n time ", Round[t, 0.001], "s numeric=", numOK, " exact=", exactOK,
33      " depth ", depthIn, "->", depthOut, If[msgs != {}, " MSGS: " <> ToString[msgs, ""]];
34    out];
35
36  Print["Kernel: ", $Version];
37  Print["Print attributes before load: ", $printAttrsBefore, " after: ", $printAttrsAfter];

```

Reproduction. With Wolfram 15 and wolframscript on the path, from `src/code-review/unified/harness`:

```

wolframscript -file exp_original.wl      # Tables 6-10
wolframscript -file exp_helpers.wl      # Tables 11-12
wolframscript -file exp_round2.wl      # Tables 13-14
wolframscript -file exp_round3.wl      # Table 15
wolframscript -file exp_fuzz.wl        # Table 16, original
wolframscript -file exp_fixed.wl      # Tables 18-22
wolframscript -file exp_fuzz_fixed.wl  # Table 16, corrected
wolframscript -file test_review_kernels.wl
./run_suites.sh                        # the reviews' own suites

```

```
wolframscript -file export_tables.wl      # regenerates tables/*.tex
```

Only one kernel should run at a time on a single-seat license. Loading the original file in a valuable interactive session is not advisable: it clears several global symbols and unprotects **Print**.